

## INTRODUCTION TO CLASSIFICATION

Databases contain a lot of hidden information which has to be retrieved using some techniques for making some business decisions.

These techniques include,

(1) Classification.

(2) Prediction.

They are used to extract models describing important data classes or to predict future data trends. Classification predicts categorical labels. For example, a classification model may be built to categorize bank loan applications as either safe or risky. Some of the classification techniques include decision tree induction, Bayesian classification, Bayesian belief network and case-based reasoning etc.

Prediction can model the continuous-valued functions. For example, in a Bank loan, a prediction model may be built to predict the expenditure of potential customers on computer equipment given their income and occupation. Methods for prediction includes linear, non-linear and generalized linear regression models.

Many classification and prediction methods have been proposed by researchers in machine learning expert systems, statistics, and neurobiology.

### **Classification**

The classification can be defined in any of the following ways,

"Classification is a Machine learning technique, Its used to predict group Membership for data instances.

(or)

"A subdivision of a set of examples into a number of classes"

(or)

"Classification is learning a function that determines a data items into one of several predefined classes."

Types : Estimation and prediction may be viewed as types of classification.

Use : The use of classification is for credit card purchases.

A large database has huge amount of raw data, which is analyzed and predicted to retrieve useful information and to make decisions. Classification is one of the methods used for data analysis. We analyze the data and classify it, based on our requirement. Example : If we want to know the performance of the university, we classify the students database based on their performance as above average, average and below average students. If the classification shows that the number of students under 'below average' category are more, then the university needs to improve.

### **General Approach to Classification**

Data classification is done in two steps. They are,

(1) Learning: In this step, a classifier is built. It is defined based on the 'training set'. The database tuples and their associated classes are called training set. Each database tuple is based on the database attribute.

Database attributes that determine the class to which other attributes belong is known as 'class label attributes'. This step is also called as supervised learning. This is because we have a clear map between the class and its attributes. This mapping is represented as  $y=f(x)$  where  $y$  is the class for the tuple  $x$ . The mapping is represented as classification rules or decision trees.

(2) Accuracy of Classification: After obtaining the desired mapping, classification is done. The accuracy of classification is now calculated. It is the percentage of tuples that are correctly classified. If this classification is accurate, we proceed further to classify data tuples for which associated class label is not found. Unsupervised learning or clustering is the one for which the class labels of the attribute cannot be determined.

Example: Let us consider the data classification where a decision is to be made for increasing the pay scale of employees in an organization based on their performance levels and current pay scale Fig. 6.2.1 and 6.2.2.

| Training Data |             |             |                   |               |
|---------------|-------------|-------------|-------------------|---------------|
| Name          | Age         | Performance | Current_pay_scale | Like_decision |
| Salman        | Young       | Good        | 20,000            | Yes           |
| Zuhair        | Young       | Good        | 25,000            | Yes           |
| Shujath       | Senior      | Good        | 26,000            | Yes           |
| Hasan         | Middle-aged | Bad         | 30,000            | No            |
| ...           | ...         | ...         | ..                | ..            |

Classification Algorithm

| Rules of Classification                                                       |
|-------------------------------------------------------------------------------|
| IF Performance = Bad THEN Like_decision = No                                  |
| IF Current_pay_scale < 26,000 and Performance = Good THEN Like_decision = Yes |
| IF Age = Senior and Current_pay_scale < 30,000 THEN Like_decision = Yes       |

**Fig. 6.21** Learning

| Test Data |             |             |                   |               |
|-----------|-------------|-------------|-------------------|---------------|
| Name      | Age         | Performance | Current_pay_scale | Like_decision |
| Raju      | Young       | Good        | 20,000            | Yes           |
| Yousuf    | Senior      | Good        | 28,000            | Yes           |
| Amjad     | Senior      | Good        | 22,000            | Yes           |
| Srikanth  | Middle-aged | Bad         | 24,000            | No            |
| ..        | ..          | ..          | ...               | ...           |

Rules of Classification

| New Data                                 |
|------------------------------------------|
| Khan, Young, Good, 20,000 Like_decision? |

Yes (Increase Pay)

## Decision tree induction

Decision tree induction is a popular method used for both classification and regression tasks in machine learning. It involves creating a model that predicts the value of a target variable by learning simple decision rules inferred from the data features.

### Basic Structure of a Decision Tree

**Nodes:** Represent features (or attributes) of the data.

**Root Node:** The top node representing the feature that best splits the data.

**Internal Nodes:** Nodes that represent features and have child nodes.

**Leaf Nodes:** Terminal nodes that represent the outcome or class label.

**Edges:** Connect nodes and represent the decision rules or conditions.

### Splitting Criteria

The core of decision tree induction is deciding how to split the data at each node. The goal is to find the split that best separates the data according to the target variable. Common splitting criteria include:

**Gini Impurity:** Measures the impurity or diversity of a dataset. It is used by the CART (Classification and Regression Tree) algorithm. For a binary classification, the Gini impurity is:

$$Gini(p) = 1 - p^2 - (1-p)^2$$

Where  $p$  is the probability of a certain class.

**Entropy and Information Gain:** Entropy measures the randomness in the data, and information gain measures the reduction in entropy after a dataset is split on an attribute. For a binary classification, entropy

$$H \text{ is calculated as: } H(p) = -p \log_2(p) - (1-p) \log_2(1-p)$$

Information gain is the difference between the entropy of the original dataset and the weighted entropy of the subsets:

$$IG(D, A) = H(D) - \sum_{v \in \text{Values}(A)} \frac{|D_v|}{|D|} H(D_v)$$

$$IG(D, A) = H(D) - \sum_{v \in \text{Values}(A)} \frac{|D_v|}{|D|} H(D_v)$$

where

$D$

$D$  is the dataset,

$A$

$A$  is the attribute, and

$D_v$  are the subsets of

$D$

$D$  after the split.

**Variance Reduction:** Used for regression trees, variance reduction measures the decrease in variance after a split.

### Decision Tree Induction Algorithm

**Select the Best Split:** Choose the feature and the value that results in the best split according to the chosen criterion (Gini impurity, information gain, etc.).

**Create Subtrees:** Recursively apply the splitting process to the resulting subsets.

**Stopping Criteria:** Stop splitting when:

- A maximum tree depth is reached.
- A minimum number of samples per node is reached.
- No further information gain is achieved.
- All samples in a node belong to the same class.

**Pruning**

To avoid overfitting, decision trees can be pruned by removing branches that have little importance. Pruning can be done in two ways:

**Pre-Pruning (Early Stopping):** Stops the tree building process early based on pre-defined criteria (like maximum depth, minimum samples per node).

**Post-Pruning:** Removes branches from a fully grown tree that do not provide significant power to classify instances. This can be done by:

Cost Complexity Pruning (CCP): Balances the tree size and its error rate.

Reduced Error Pruning: Prunes nodes if the tree's accuracy on a validation set does not decrease.

### **Advantages and Disadvantages**

#### **Advantages:**

Simple to understand and interpret.

Requires little data preprocessing.

Handles both numerical and categorical data.

Can capture non-linear relationships.

#### **Disadvantages:**

Prone to overfitting, especially with deep trees.

Can be unstable (small variations in data might lead to different trees).

Does not perform well on datasets with many irrelevant features.

## Iterative Dichotomiser 3

ID3 stands for Iterative Dichotomiser 3 and is named after that because the algorithm iteratively (repeatedly) dichotomizes(divides) features into two or more groups at each step. Invented by Ross Quinlan, ID3 uses a top-down greedy approach to build a decision tree. In simple words, the top-down approach means that we start building the tree from the top and the greedy approach means that at each iteration we select the best feature at the present moment to create a node.

### ID3 Steps

- Calculate the Information Gain of each feature.
- Considering that all rows don't belong to the same class, split the dataset S into subsets using the feature for which the Information Gain is maximum.
- Make a decision tree node using the feature with the maximum Information gain.
- If all rows belong to the same class, make the current node as a leaf node with the class as its label.
- Repeat for the remaining features until we run out of all features, or the decision tree has all leaf nodes.

### What is Entropy and Information gain?

Entropy is a measure of the amount of uncertainty in the dataset S. Mathematical Representation of Entropy is shown here -

$$H(S) = - \sum_{c \in C} p(c) \log_2 p(c)$$

Where,

- S - The current dataset for which entropy is being calculated(changes every iteration of the ID3 algorithm).
- C - Set of classes in S {example - C={yes, no}}
- p(c) - The proportion of the number of elements in class c to the number of elements in set S.



In ID3, entropy is calculated for each remaining attribute. The attribute with the smallest entropy is used to split the set S on that particular iteration. Entropy = 0 implies it is of pure class, that means all are of same category.

Information Gain  $IG(A)$  tells us how much uncertainty in S was reduced after splitting set S on attribute A. Mathematical representation of Information gain is shown here -

$$IG(A, S) = H(S) - \sum_{t \in T} p(t) H(t)$$

Where,

- $H(S)$  - Entropy of set S.
- T - The subsets created from splitting set S by attribute A such that

$$S = \bigcup_{t \in T} t$$

- $p(t)$  - The proportion of the number of elements in t to the number of elements in set S.
- $H(t)$  - Entropy of subset t.

In ID3, information gain can be calculated (instead of entropy) for each remaining attribute. The attribute with the largest information gain is used to split the set S on that particular iteration.

### Use ID3 algorithm on a data

| Day | Outlook  | Temperature | Humidity | Wind   | Play Tennis |
|-----|----------|-------------|----------|--------|-------------|
| 1   | Sunny    | Hot         | High     | Weak   | No          |
| 2   | Sunny    | Hot         | High     | Strong | No          |
| 3   | Overcast | Hot         | High     | Weak   | Yes         |
| 4   | Rain     | Mild        | High     | Weak   | Yes         |
| 5   | Rain     | Cool        | Normal   | Weak   | Yes         |
| 6   | Rain     | Cool        | Normal   | Strong | No          |
| 7   | Overcast | Cool        | Normal   | Strong | Yes         |
| 8   | Sunny    | Mild        | High     | Weak   | No          |
| 9   | Sunny    | Cool        | Normal   | Weak   | Yes         |
| 10  | Rain     | Mild        | Normal   | Weak   | Yes         |
| 11  | Sunny    | Mild        | Normal   | Strong | Yes         |
| 12  | Overcast | Mild        | High     | Strong | Yes         |
| 13  | Overcast | Hot         | Normal   | Weak   | Yes         |
| 14  | Rain     | Mild        | High     | Strong | No          |

Here, dataset is of binary classes (yes and no), where 9 out of 14 are "yes" and 5 out of 14 are "no".

Complete entropy of dataset is -

$$\begin{aligned}
 H(S) &= -p(\text{yes}) * \log_2(p(\text{yes})) - p(\text{no}) * \log_2(p(\text{no})) \\
 &= -(9/14) * \log_2(9/14) - (5/14) * \log_2(5/14)
 \end{aligned}$$

$$= -(-0.41) - (-0.53)$$

$$= 0.94$$

For each attribute of the dataset, let's follow the step-2 of pseudocode : -

#### First Attribute - Outlook

Categorical values - sunny, overcast and rain

$$H(\text{Outlook}=\text{sunny}) = -(2/5)*\log(2/5)-(3/5)*\log(3/5) = 0.971$$

$$H(\text{Outlook}=\text{rain}) = -(3/5)*\log(3/5)-(2/5)*\log(2/5) = 0.971$$

$$H(\text{Outlook}=\text{overcast}) = -(4/4)*\log(4/4) - 0 = 0$$

Average Entropy Information for Outlook -

$$I(\text{Outlook}) = p(\text{sunny}) * H(\text{Outlook}=\text{sunny}) + p(\text{rain}) * H(\text{Outlook}=\text{rain}) + p(\text{overcast}) * H(\text{Outlook}=\text{overcast})$$

$$= (5/14)*0.971 + (5/14)*0.971 + (4/14)*0$$

$$= 0.693$$

$$\text{Information Gain} = H(S) - I(\text{Outlook})$$

$$= 0.94 - 0.693$$

$$= 0.247$$

#### Second Attribute - Temperature

Categorical values - hot, mild, cool

$$H(\text{Temperature}=\text{hot}) = -(2/4)*\log(2/4)-(2/4)*\log(2/4) = 1$$

$$H(\text{Temperature}=\text{cool}) = -(3/4)*\log(3/4)-(1/4)*\log(1/4) = 0.811$$

$$H(\text{Temperature}=\text{mild}) = -(4/6)*\log(4/6)-(2/6)*\log(2/6) = 0.9179$$

Average Entropy Information for Temperature -

$$I(\text{Temperature}) = p(\text{hot})*H(\text{Temperature}=\text{hot}) + p(\text{mild})*H(\text{Temperature}=\text{mild}) + p(\text{cool})*H(\text{Temperature}=\text{cool})$$

$$= (4/14)*1 + (6/14)*0.9179 + (4/14)*0.811$$

$$= 0.9108$$

$$\text{Information Gain} = H(S) - I(\text{Temperature})$$

$$= 0.94 - 0.9108$$

$$= 0.0292$$

#### Third Attribute - Humidity

Categorical values - high, normal

$$H(\text{Humidity}=\text{high}) = -(3/7)*\log(3/7)-(4/7)*\log(4/7) = 0.983$$

$$H(\text{Humidity}=\text{normal}) = -(6/7)*\log(6/7)-(1/7)*\log(1/7) = 0.591$$

Average Entropy Information for Humidity -

$$I(\text{Humidity}) = p(\text{high})*H(\text{Humidity}=\text{high}) + p(\text{normal})*H(\text{Humidity}=\text{normal})$$

$$= (7/14)*0.983 + (7/14)*0.591$$

$$= 0.787$$

$$\text{Information Gain} = H(S) - I(\text{Humidity})$$

$$= 0.94 - 0.787$$

$$= 0.153$$

Fourth Attribute - Wind

Categorical values - weak, strong

$$H(\text{Wind}=\text{weak}) = -(6/8)*\log(6/8)-(2/8)*\log(2/8) = 0.811$$

$$H(\text{Wind}=\text{strong}) = -(3/6)*\log(3/6)-(3/6)*\log(3/6) = 1$$

Average Entropy Information for Wind -

$$I(\text{Wind}) = p(\text{weak})*H(\text{Wind}=\text{weak}) + p(\text{strong})*H(\text{Wind}=\text{strong})$$

$$= (8/14)*0.811 + (6/14)*1$$

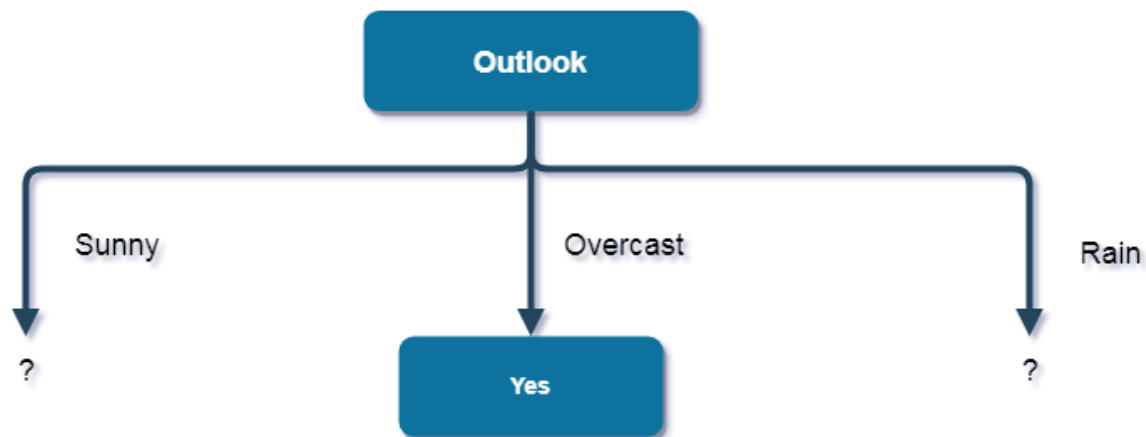
$$= 0.892$$

$$\text{Information Gain} = H(S) - I(\text{Wind})$$

$$= 0.94 - 0.892$$

$$= 0.048$$

Here, the attribute with maximum information gain is Outlook. So, the decision tree built so far -



Here, when Outlook == overcast, it is of pure class(Yes). Now, we have to repeat same procedure for the data with rows consist of Outlook value as Sunny and then for Outlook value as Rain.

Now, finding the best attribute for splitting the data with Outlook=Sunny values{ Dataset rows = [1, 2, 8, 9, 11]}.

Complete entropy of Sunny is -

$$H(S) = - p(\text{yes}) * \log_2(p(\text{yes})) - p(\text{no}) * \log_2(p(\text{no}))$$

$$= - (2/5) * \log_2(2/5) - (3/5) * \log_2(3/5)$$

$$= 0.971$$

First Attribute - Temperature

Categorical values - hot, mild, cool

$$H(\text{Sunny, Temperature}=\text{hot}) = -0-(2/2)*\log(2/2) = 0$$

$$H(\text{Sunny, Temperature}=\text{cool}) = -(1)*\log(1)- 0 = 0$$

$$H(\text{Sunny, Temperature=mild}) = -(1/2)*\log(1/2)-(1/2)*\log(1/2) = 1$$

Average Entropy Information for Temperature -

$$\begin{aligned} I(\text{Sunny, Temperature}) &= p(\text{Sunny, hot}) * H(\text{Sunny, Temperature=hot}) + p(\text{Sunny, mild}) * H(\text{Sunny, Temperature=mild}) + p(\text{Sunny, cool}) * H(\text{Sunny, Temperature=cool}) \\ &= (2/5)*0 + (1/5)*0 + (2/5)*1 \\ &= 0.4 \end{aligned}$$

$$\begin{aligned} \text{Information Gain} &= H(\text{Sunny}) - I(\text{Sunny, Temperature}) \\ &= 0.971 - 0.4 \\ &= 0.571 \end{aligned}$$

Second Attribute - Humidity

Categorical values - high, normal

$$H(\text{Sunny, Humidity=high}) = -0 - (3/3)*\log(3/3) = 0$$

$$H(\text{Sunny, Humidity=normal}) = -(2/2)*\log(2/2) - 0 = 0$$

Average Entropy Information for Humidity -

$$\begin{aligned} I(\text{Sunny, Humidity}) &= p(\text{Sunny, high}) * H(\text{Sunny, Humidity=high}) + p(\text{Sunny, normal}) * H(\text{Sunny, Humidity=normal}) \\ &= (3/5)*0 + (2/5)*0 \\ &= 0 \end{aligned}$$

$$\begin{aligned} \text{Information Gain} &= H(\text{Sunny}) - I(\text{Sunny, Humidity}) \\ &= 0.971 - 0 \\ &= 0.971 \end{aligned}$$

Third Attribute - Wind

Categorical values - weak, strong

$$H(\text{Sunny, Wind=weak}) = -(1/3)*\log(1/3)-(2/3)*\log(2/3) = 0.918$$

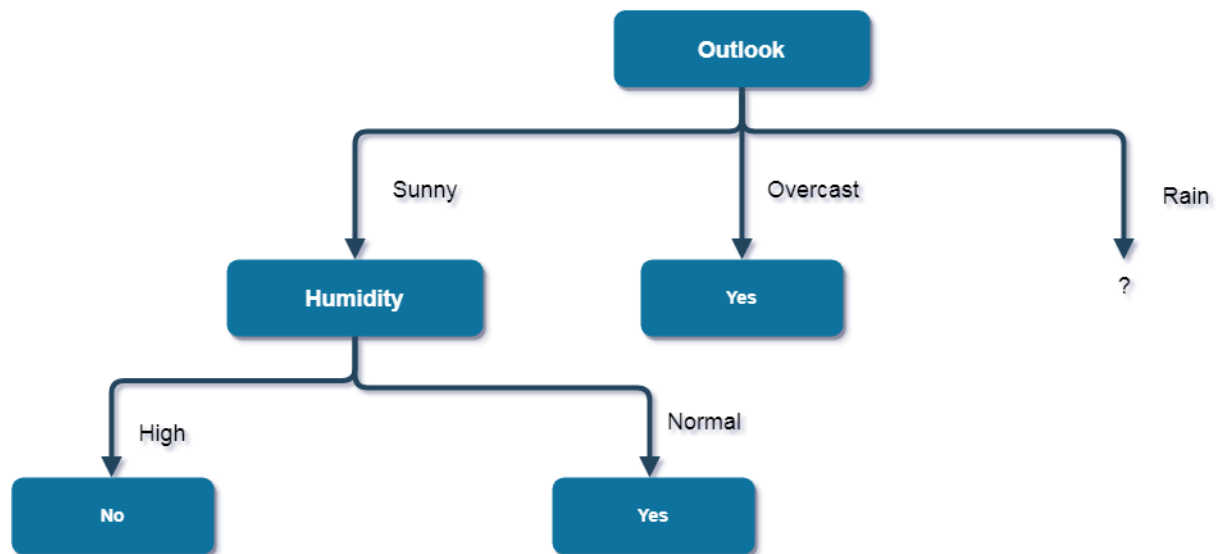
$$H(\text{Sunny, Wind=strong}) = -(1/2)*\log(1/2)-(1/2)*\log(1/2) = 1$$

Average Entropy Information for Wind -

$$\begin{aligned} I(\text{Sunny, Wind}) &= p(\text{Sunny, weak}) * H(\text{Sunny, Wind=weak}) + p(\text{Sunny, strong}) * H(\text{Sunny, Wind=strong}) \\ &= (3/5)*0.918 + (2/5)*1 \\ &= 0.9508 \end{aligned}$$

$$\begin{aligned} \text{Information Gain} &= H(\text{Sunny}) - I(\text{Sunny, Wind}) \\ &= 0.971 - 0.9508 \\ &= 0.0202 \end{aligned}$$

Here, the attribute with maximum information gain is Humidity. So, the decision tree built so far -



Here, when Outlook = Sunny and Humidity = High, it is a pure class of category "no". And When Outlook = Sunny and Humidity = Normal, it is again a pure class of category "yes". Therefore, we don't need to do further calculations.

Now, finding the best attribute for splitting the data with Outlook=Sunny values { Dataset rows = [4, 5, 6, 10, 14]}.

Complete entropy of Rain is -

$$\begin{aligned}
 H(S) &= -p(\text{yes}) * \log_2(p(\text{yes})) - p(\text{no}) * \log_2(p(\text{no})) \\
 &= - (3/5) * \log(3/5) - (2/5) * \log(2/5) \\
 &= 0.971
 \end{aligned}$$

First Attribute - Temperature

Categorical values - mild, cool

$$\begin{aligned}
 H(\text{Rain}, \text{Temperature}=\text{cool}) &= -(1/2)*\log(1/2) - (1/2)*\log(1/2) = 1 \\
 H(\text{Rain}, \text{Temperature}=\text{mild}) &= -(2/3)*\log(2/3) - (1/3)*\log(1/3) = 0.918
 \end{aligned}$$

Average Entropy Information for Temperature -

$$\begin{aligned}
 I(\text{Rain}, \text{Temperature}) &= p(\text{Rain}, \text{mild}) * H(\text{Rain}, \text{Temperature}=\text{mild}) + p(\text{Rain}, \text{cool}) * H(\text{Rain}, \text{Temperature}=\text{cool}) \\
 &= (2/5) * 1 + (3/5) * 0.918 \\
 &= 0.9508
 \end{aligned}$$

$$\begin{aligned}
 \text{Information Gain} &= H(\text{Rain}) - I(\text{Rain}, \text{Temperature}) \\
 &= 0.971 - 0.9508 \\
 &= 0.0202
 \end{aligned}$$

Second Attribute - Wind

Categorical values - weak, strong

$$\begin{aligned}
 H(\text{Wind}=\text{weak}) &= -(3/3)*\log(3/3) - 0 = 0 \\
 H(\text{Wind}=\text{strong}) &= 0 - (2/2)*\log(2/2) = 0
 \end{aligned}$$

Average Entropy Information for Wind -

$$I(\text{Wind}) = p(\text{Rain, weak}) * H(\text{Rain, Wind=weak}) + p(\text{Rain, strong}) * H(\text{Rain, Wind=strong})$$

$$= (3/5) * 0 + (2/5) * 0$$

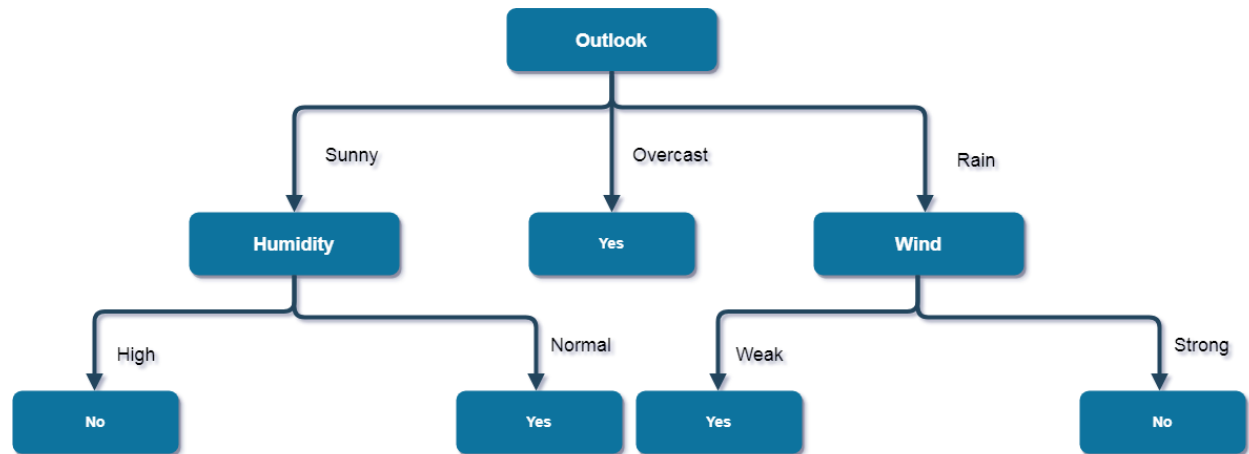
$$= 0$$

$$\text{Information Gain} = H(\text{Rain}) - I(\text{Rain, Wind})$$

$$= 0.971 - 0$$

$$= 0.971$$

Here, the attribute with maximum information gain is Wind. So, the decision tree built so far



Here, when Outlook = Rain and Wind = Strong, it is a pure class of category "no". And When Outlook = Rain and Wind = Weak, it is again a pure class of category "yes". And this is our final desired tree for the given dataset.

# Classification and Regression Trees

The CART (Classification and Regression Trees) algorithm is a decision tree-based machine learning technique used for both classification and regression tasks.

Tree Structure: CART constructs a binary tree structure where each internal node represents a decision based on a feature's value, and each leaf node corresponds to a class label

Splitting Criteria: typically measured by metrics like Gini impurity for classification or mean squared error for regression.

Pruning: After constructing the tree, pruning is performed to avoid overfitting.

| Day | Outlook  | Temp. | Humidity | Wind   | Decision |
|-----|----------|-------|----------|--------|----------|
| 1   | Sunny    | Hot   | High     | Weak   | No       |
| 2   | Sunny    | Hot   | High     | Strong | No       |
| 3   | Overcast | Hot   | High     | Weak   | Yes      |
| 4   | Rain     | Mild  | High     | Weak   | Yes      |
| 5   | Rain     | Cool  | Normal   | Weak   | Yes      |
| 6   | Rain     | Cool  | Normal   | Strong | No       |
| 7   | Overcast | Cool  | Normal   | Strong | Yes      |

|    |          |      |        |        |     |
|----|----------|------|--------|--------|-----|
| 8  | Sunny    | Mild | High   | Weak   | No  |
| 9  | Sunny    | Cool | Normal | Weak   | Yes |
| 10 | Rain     | Mild | Normal | Weak   | Yes |
| 11 | Sunny    | Mild | Normal | Strong | Yes |
| 12 | Overcast | Mild | High   | Strong | Yes |
| 13 | Overcast | Hot  | Normal | Weak   | Yes |
| 14 | Rain     | Mild | High   | Strong | No  |

### Gini index

Gini index is a metric for classification tasks in CART. It stores sum of squared probabilities of each class. We can formulate it as illustrated below.

$$\text{Gini} = 1 - \sum (P_i)^2 \text{ for } i=1 \text{ to number of classes}$$

### Outlook

Outlook is a nominal feature. It can be sunny, overcast or rain. I will summarize the final decisions for outlook feature.

|         |     |    |                     |
|---------|-----|----|---------------------|
| Outlook | Yes | No | Number of instances |
|---------|-----|----|---------------------|



|          |   |   |   |
|----------|---|---|---|
| Sunny    | 2 | 3 | 5 |
| Overcast | 4 | 0 | 4 |
| Rain     | 3 | 2 | 5 |

$$\text{Gini}(\text{Outlook}=\text{Sunny}) = 1 - (2/5)^2 - (3/5)^2 = 1 - 0.16 - 0.36 = 0.48$$

$$\text{Gini}(\text{Outlook}=\text{Overcast}) = 1 - (4/4)^2 - (0/4)^2 = 0$$

$$\text{Gini}(\text{Outlook}=\text{Rain}) = 1 - (3/5)^2 - (2/5)^2 = 1 - 0.36 - 0.16 = 0.48$$

Then, we will calculate weighted sum of gini indexes for outlook feature.

$$\text{Gini}(\text{Outlook}) = (5/14) \times 0.48 + (4/14) \times 0 + (5/14) \times 0.48 = 0.171 + 0 + 0.171 = 0.342$$

### Temperature

Similarly, temperature is a nominal feature and it could have 3 different values: Cool, Hot and Mild. Let's summarize decisions for temperature feature.

| Temperature | Yes | No | Number of instances |
|-------------|-----|----|---------------------|
| Hot         | 2   | 2  | 4                   |
| Cool        | 3   | 1  | 4                   |
| Mild        | 4   | 2  | 6                   |

$$\text{Gini}(\text{Temp}=\text{Hot}) = 1 - (2/4)^2 - (2/4)^2 = 0.5$$

$$\text{Gini}(\text{Temp}=\text{Cool}) = 1 - (3/4)^2 - (1/4)^2 = 1 - 0.5625 - 0.0625 = 0.375$$

$$\text{Gini}(\text{Temp}=\text{Mild}) = 1 - (4/6)^2 - (2/6)^2 = 1 - 0.444 - 0.111 = 0.445$$

We'll calculate weighted sum of gini index for temperature feature

$$\text{Gini}(\text{Temp}) = (4/14) \times 0.5 + (4/14) \times 0.375 + (6/14) \times 0.445 = 0.142 + 0.107 + 0.190 = 0.439$$

## Humidity

Humidity is a binary class feature. It can be high or normal.

| Humidity | Yes | No | Number of instances |
|----------|-----|----|---------------------|
| High     | 3   | 4  | 7                   |
| Normal   | 6   | 1  | 7                   |

$$\text{Gini}(\text{Humidity}=\text{High}) = 1 - (3/7)^2 - (4/7)^2 = 1 - 0.183 - 0.326 = 0.489$$

$$\text{Gini}(\text{Humidity}=\text{Normal}) = 1 - (6/7)^2 - (1/7)^2 = 1 - 0.734 - 0.02 = 0.244$$

Weighted sum for humidity feature will be calculated next

$$\text{Gini}(\text{Humidity}) = (7/14) \times 0.489 + (7/14) \times 0.244 = 0.367$$

## Wind

Wind is a binary class similar to humidity. It can be weak and strong.

| Wind   | Yes | No | Number of instances |
|--------|-----|----|---------------------|
| Weak   | 6   | 2  | 8                   |
| Strong | 3   | 3  | 6                   |

$$\text{Gini}(\text{Wind}=\text{Weak}) = 1 - (6/8)^2 - (2/8)^2 = 1 - 0.5625 - 0.0625 = 0.375$$

$$\text{Gini}(\text{Wind}=\text{Strong}) = 1 - (3/6)^2 - (3/6)^2 = 1 - 0.25 - 0.25 = 0.5$$

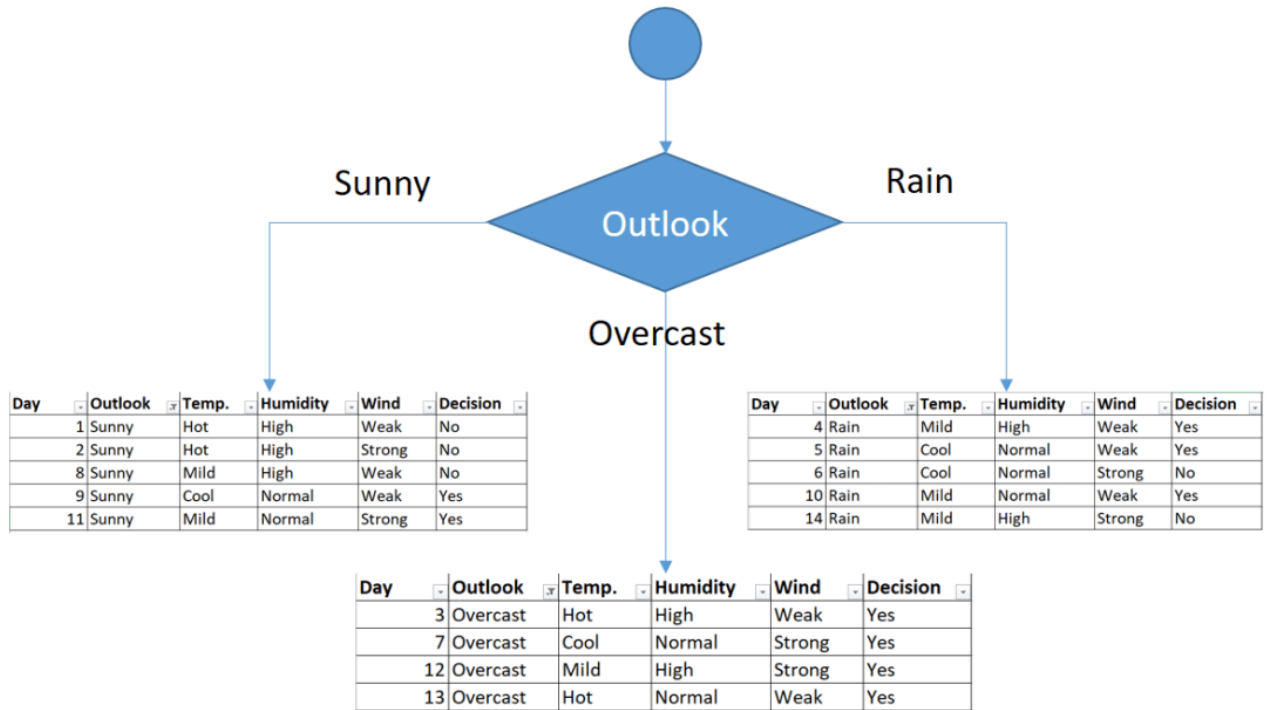
$$\text{Gini}(\text{Wind}) = (8/14) \times 0.375 + (6/14) \times 0.5 = 0.428$$

### Time to decide

We've calculated gini index values for each feature. The winner will be outlook feature because its cost is the lowest.

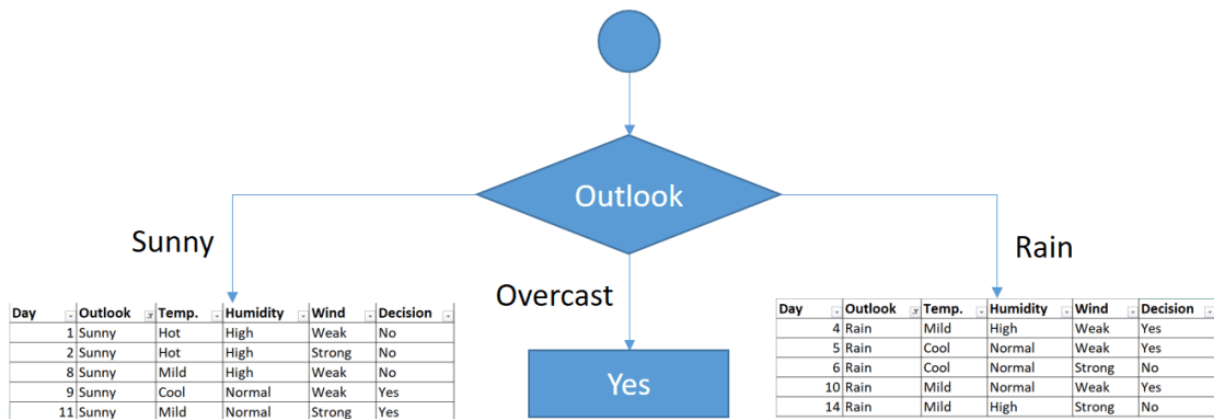
| Feature     | Gini index |
|-------------|------------|
| Outlook     | 0.342      |
| Temperature | 0.439      |
| Humidity    | 0.367      |
| Wind        | 0.428      |

We'll put outlook decision at the top of the tree.



First decision would be outlook feature

You might realize that sub dataset in the overcast leaf has only yes decisions. This means that overcast leaf is over.



Tree is over for overcast outlook leaf

We will apply same principles to those sub datasets in the following steps.

Focus on the sub dataset for sunny outlook. We need to find the gini index scores for temperature, humidity and wind features respectively.

| Day | Outlook | Temp. | Humidity | Wind   | Decision |
|-----|---------|-------|----------|--------|----------|
| 1   | Sunny   | Hot   | High     | Weak   | No       |
| 2   | Sunny   | Hot   | High     | Strong | No       |
| 8   | Sunny   | Mild  | High     | Weak   | No       |
| 9   | Sunny   | Cool  | Normal   | Weak   | Yes      |
| 11  | Sunny   | Mild  | Normal   | Strong | Yes      |

#### Gini of temperature for sunny outlook

| Temperature | Yes | No | Number of instances |
|-------------|-----|----|---------------------|
| Hot         | 0   | 2  | 2                   |
| Cool        | 1   | 0  | 1                   |
| Mild        | 1   | 1  | 2                   |

$$\text{Gini}(\text{Outlook}=\text{Sunny and Temp.}=\text{Hot}) = 1 - (0/2)^2 - (2/2)^2 = 0$$

$$\text{Gini}(\text{Outlook}=\text{Sunny and Temp.}=\text{Cool}) = 1 - (1/1)^2 - (0/1)^2 = 0$$

$$\text{Gini}(\text{Outlook}=\text{Sunny and Temp.}=\text{Mild}) = 1 - (1/2)^2 - (1/2)^2 = 1 - 0.25 - 0.25 = 0.5$$

$$\text{Gini}(\text{Outlook}=\text{Sunny and Temp.}) = (2/5) \times 0 + (1/5) \times 0 + (2/5) \times 0.5 = 0.2$$

**Gini of humidity for sunny outlook**

| Humidity | Yes | No | Number of instances |
|----------|-----|----|---------------------|
| High     | 0   | 3  | 3                   |
| Normal   | 2   | 0  | 2                   |

$$\text{Gini}(\text{Outlook}=\text{Sunny and Humidity}=\text{High}) = 1 - (0/3)^2 - (3/3)^2 = 0$$

$$\text{Gini}(\text{Outlook}=\text{Sunny and Humidity}=\text{Normal}) = 1 - (2/2)^2 - (0/2)^2 = 0$$

$$\text{Gini}(\text{Outlook}=\text{Sunny and Humidity}) = (3/5) \times 0 + (2/5) \times 0 = 0$$

**Gini of wind for sunny outlook**

| Wind   | Yes | No | Number of instances |
|--------|-----|----|---------------------|
| Weak   | 1   | 2  | 3                   |
| Strong | 1   | 1  | 2                   |

$$\text{Gini}(\text{Outlook}=\text{Sunny and Wind}=\text{Weak}) = 1 - (1/3)^2 - (2/3)^2 = 0.266$$

$$\text{Gini}(\text{Outlook}=\text{Sunny and Wind}=\text{Strong}) = 1 - (1/2)^2 - (1/2)^2 = 0.2$$

$$\text{Gini}(\text{Outlook}=\text{Sunny and Wind}) = (3/5) \times 0.266 + (2/5) \times 0.2 = 0.466$$

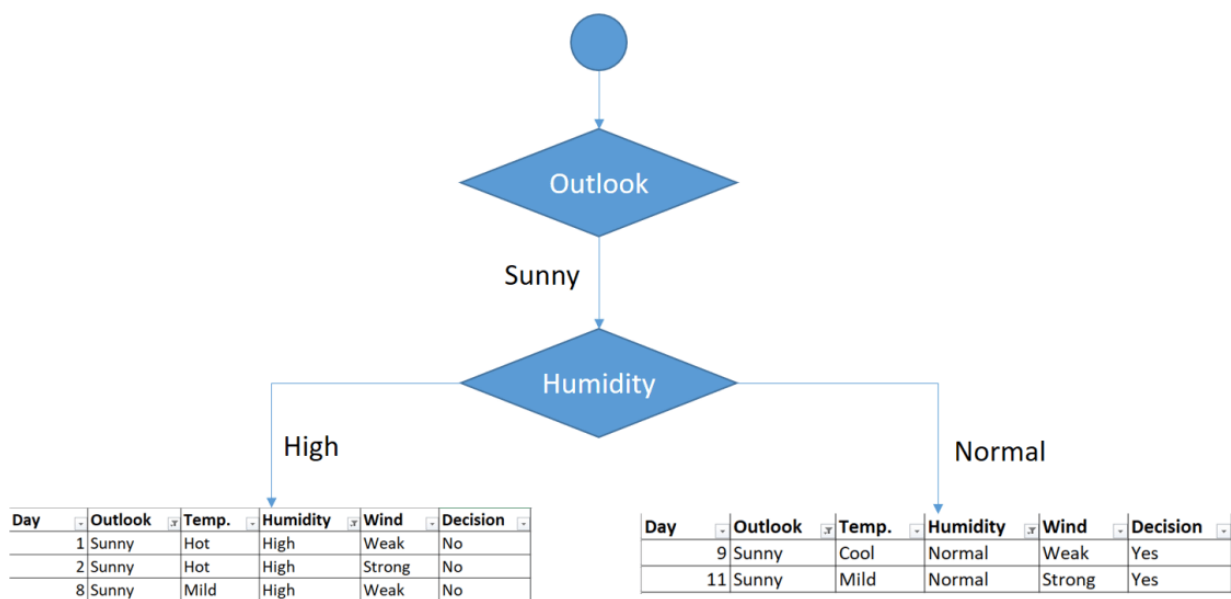
**Decision for sunny outlook**

We've calculated gini index scores for feature when outlook is sunny. The winner is humidity because it has the lowest value.

| Feature | Gini index |
|---------|------------|
|         |            |

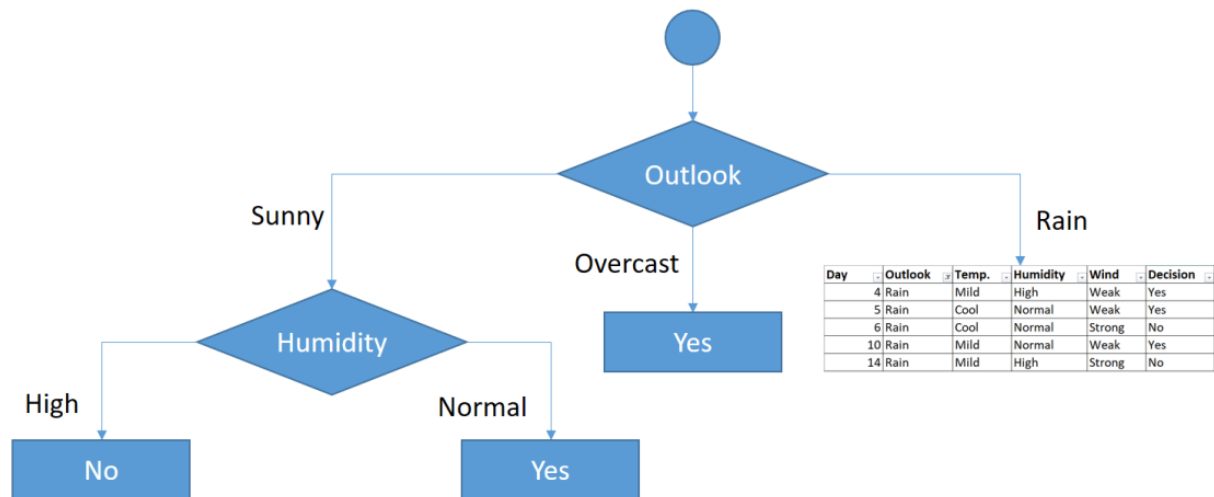
|             |       |
|-------------|-------|
| Temperature | 0.2   |
| Humidity    | 0     |
| Wind        | 0.466 |

We'll put humidity check at the extension of sunny outlook.



### Sub datasets for high and normal humidity

As seen, decision is always no for high humidity and sunny outlook. On the other hand, decision will always be yes for normal humidity and sunny outlook. This branch is over.



Decisions for high and normal humidity  
Now, we need to focus on rain outlook.

### Rain outlook

| Day | Outlook | Temp. | Humidity | Wind   | Decision |
|-----|---------|-------|----------|--------|----------|
| 4   | Rain    | Mild  | High     | Weak   | Yes      |
| 5   | Rain    | Cool  | Normal   | Weak   | Yes      |
| 6   | Rain    | Cool  | Normal   | Strong | No       |
| 10  | Rain    | Mild  | Normal   | Weak   | Yes      |
| 14  | Rain    | Mild  | High     | Strong | No       |

We'll calculate gini index scores for temperature, humidity and wind features when outlook is rain.



**Gini of temprature for rain outlook**

| Temperature | Yes | No | Number of instances |
|-------------|-----|----|---------------------|
| Cool        | 1   | 1  | 2                   |
| Mild        | 2   | 1  | 3                   |

$$\text{Gini}(\text{Outlook}=\text{Rain and Temp.}=\text{Cool}) = 1 - (1/2)^2 - (1/2)^2 = 0.5$$

$$\text{Gini}(\text{Outlook}=\text{Rain and Temp.}=\text{Mild}) = 1 - (2/3)^2 - (1/3)^2 = 0.444$$

$$\text{Gini}(\text{Outlook}=\text{Rain and Temp.}) = (2/5) \times 0.5 + (3/5) \times 0.444 = 0.466$$

**Gini of humidity for rain outlook**

| Humidity | Yes | No | Number of instances |
|----------|-----|----|---------------------|
| High     | 1   | 1  | 2                   |
| Normal   | 2   | 1  | 3                   |

$$\text{Gini}(\text{Outlook}=\text{Rain and Humidity}=\text{High}) = 1 - (1/2)^2 - (1/2)^2 = 0.5$$

$$\text{Gini}(\text{Outlook}=\text{Rain and Humidity}=\text{Normal}) = 1 - (2/3)^2 - (1/3)^2 = 0.444$$

$$\text{Gini}(\text{Outlook}=\text{Rain and Humidity}) = (2/5) \times 0.5 + (3/5) \times 0.444 = 0.466$$

**Gini of wind for rain outlook**

| Wind | Yes | No | Number of instances |
|------|-----|----|---------------------|
|      |     |    |                     |

|        |   |   |   |
|--------|---|---|---|
| Weak   | 3 | 0 | 3 |
| Strong | 0 | 2 | 2 |

$$\text{Gini}(\text{Outlook}=\text{Rain and Wind}=\text{Weak}) = 1 - (3/3)^2 - (0/3)^2 = 0$$

$$\text{Gini}(\text{Outlook}=\text{Rain and Wind}=\text{Strong}) = 1 - (0/2)^2 - (2/2)^2 = 0$$

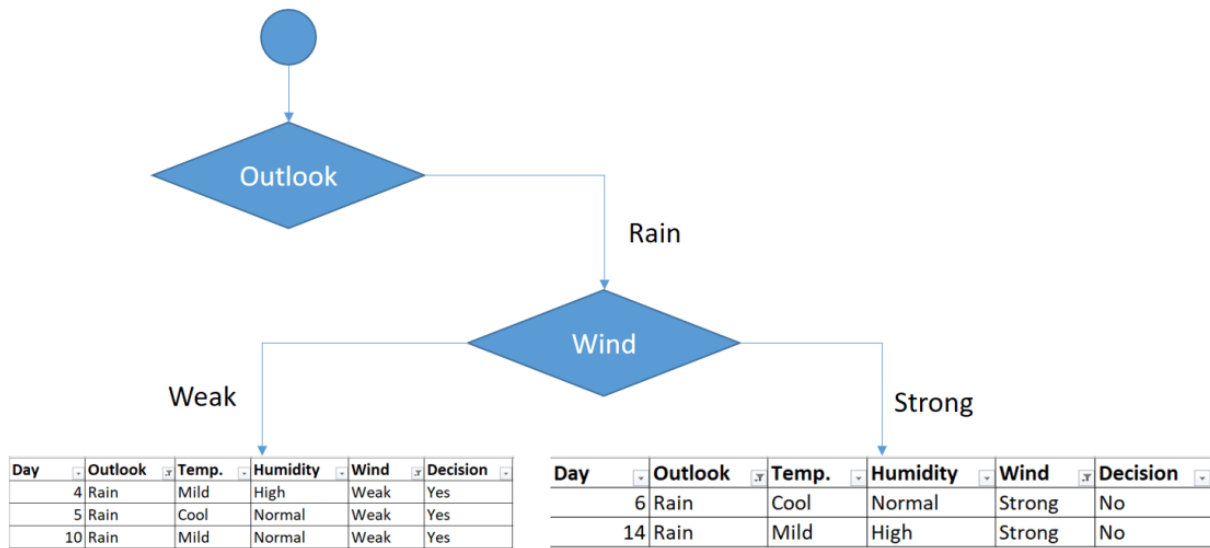
$$\text{Gini}(\text{Outlook}=\text{Rain and Wind}) = (3/5) \times 0 + (2/5) \times 0 = 0$$

### Decision for rain outlook

The winner is wind feature for rain outlook because it has the minimum gini index score in features.

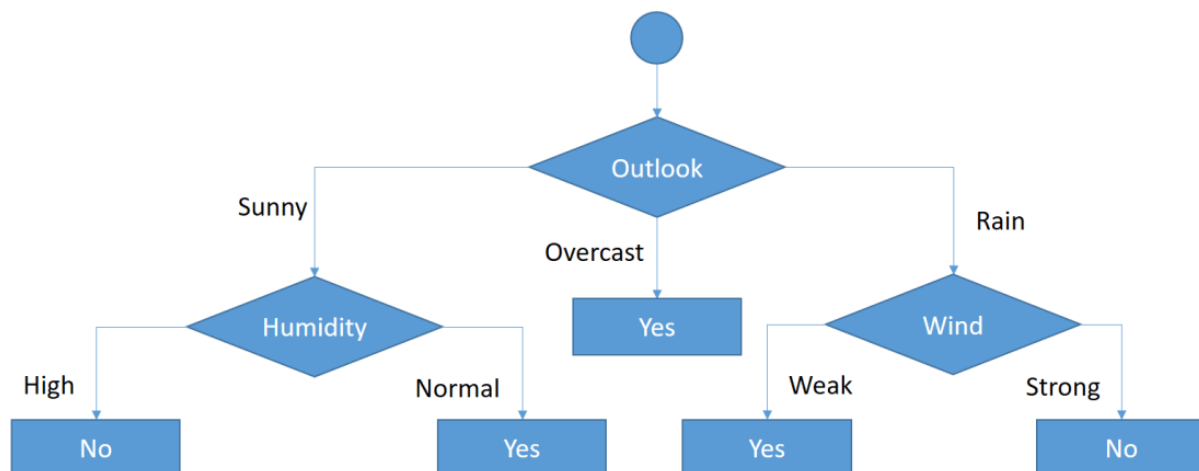
| Feature     | Gini index |
|-------------|------------|
| Temperature | 0.466      |
| Humidity    | 0.466      |
| Wind        | 0          |

Put the wind feature for rain outlook branch and monitor the new sub data sets.



Sub data sets for weak and strong wind and rain outlook

As seen, decision is always yes when wind is weak. On the other hand, decision is always no if wind is strong. This means that this branch is over.



### Probability Theory

Probability theory is a mathematical framework for quantifying our uncertainty about the world. It allows us (and our software) to reason effectively in situations where being certain is impossible. Probability theory is at the foundation of many machine learning algorithms.

The word 'Probability' means the chance of occurring of a particular event. It is generally possible to predict the future of an event quantitatively with a certain probability of being correct. The probability is used in such cases where the outcome of the trial is uncertain.

The probability of happening of an event A, denoted by  $P(A)$ , is defined as

$$P(A) = \frac{\text{number of cases favourable to A}}{\text{number of possible outcomes}}$$

Thus, if an event can happen in  $m$  ways and fails to occur in  $n$  ways and  $m+n$  ways is equally likely to occur then the probability of happening of the event A is given by

$$P(A) = \frac{m}{m+n}$$

And the probability of non-happening of A is

$$P(\overline{A}) = \frac{n}{m+n}$$

1. The probability of an event which is certain to occur is one.
2. The probability of an event which is impossible to zero.
3. If the probability of happening of an event  $P(A)$  and that of not happening is  $P(\overline{A})$ , then  $P(A) + P(\overline{A}) = 1$ ,  $0 \leq P(A) \leq 1$ ,  $0 \leq P(\overline{A}) \leq 1$ .

Important Terms related to Probability:

**Trial and Event:** The performance of an experiment is called a trial, and the set of its outcomes is termed an event.

**Random Experiment:** It is an experiment in which all the possible outcomes of the experiment are known in advance. But the exact outcomes of any specific performance are not known in advance.

**Outcome:** The result of a random experiment is called an Outcome.

**Sample Space:** The set of all possible outcomes of an experiment is called sample space and is denoted by S.

**Complement of Event:** The set of all outcomes which are in sample space but not an event is called the complement of an event.

**Impossible Events:** An event which will never be happened.

**Sure Outcome/Certain Outcome:** An Outcome which will definitely be happen

**Possible Outcome:** An outcome which is possible to occur is called Possible Outcome.

**Equally Likely Events:** Events are said to be equally likely if one of them cannot be expected to occur in preference to others. In other words, it means each outcome is as likely to occur as any other outcome.

**Mutually Exclusive or Disjoint Events:** Events are called mutually exclusive if they cannot occur simultaneously.

**Exhaustive Events:** The total number of all possible outcomes of an experiment is called exhaustive events.

**Independent Events:** Events A and B are said to be independent if the occurrence of any one event does not affect the occurrence of any other event.

$$P(A \cap B) = P(A) P(B).$$

Bayesian Learning:

- ☐ Provides practical learning algorithms
- ☐ Naïve Bayes learning
- ☐ Bayesian belief network learning
- ☐ Combine prior knowledge (prior probabilities)
- ☐ Provides foundations for machine learning
- ☐ Evaluating learning algorithms
- ☐ Guiding the design of new algorithms
- ☐ Learning from models : meta learning

Why Bayesian Classification?

Probabilistic learning: Calculate explicit probabilities for hypothesis, among the most practical approaches to certain types of learning problems

Incremental: Each training example can incrementally increase/decrease the probability that a hypothesis is correct. Prior knowledge can be combined with observed data.

Probabilistic prediction: Predict multiple hypotheses, weighted by their probabilities

Standard: Even when Bayesian methods are computationally intractable, they can provide a standard of optimal decision making against which other methods can be measured

## Bayes Theorem - Example

Sample Space for  
events A and B

|                |   |   |   |   |   |   |   |
|----------------|---|---|---|---|---|---|---|
| <i>A holds</i> | T | T | F | F | T | F | T |
| <i>B holds</i> | T | F | T | F | T | F | F |

$$P(A) = 4/7$$

$$P(B) = 3/7$$

$$P(B|A) = 2/4$$

$$P(A|B) = 2/3$$

Is Bayes Theorem correct?

$$P(B|A) = P(A|B)P(B) / P(A) = (2/3 * 3/7) / 4/7 = 2/4$$

➔ CORRECT

$$P(A|B) = P(B|A)P(A) / P(B) = (2/4 * 4/7) / 3/7 = 2/3$$

➔ CORRECT

## Basic Approach

Bayes Rule:

$$P(h | D) = \frac{P(D | h)P(h)}{P(D)}$$

- $P(h)$  = prior probability of hypothesis  $h$
- $P(D)$  = prior probability of training data  $D$
- $P(h|D)$  = probability of  $h$  given  $D$  (posterior density)
- $P(D|h)$  = probability of  $D$  given  $h$  (likelihood of  $D$  given  $h$ )

The Goal of Bayesian Learning: the most probable hypothesis given the training data (Maximum A Posteriori hypothesis  $h_{map}$ .)

$$\begin{aligned} h_{map} &= \max_{h \in H} P(h | D) \\ &= \max_{h \in H} \frac{P(D | h)P(h)}{P(D)} \\ &= \max_{h \in H} P(D | h)P(h) \\ &= \max_{h \in H} P(D | h) \end{aligned}$$

*Note that the final step above we dropped the term  $P(D)$  or  $P(h)$  as they are constants independent of the other*

### An Example: Does patient have cancer or not?

A patient takes a lab test and the result comes back positive. The test returns a correct positive result in only 98% of the cases in which the disease is actually present, and a correct negative result in only 97% of the cases in which the disease is not present. Furthermore, .008 of the entire population have this cancer.

$$P(\text{cancer}) = .008, P(\neg \text{cancer}) = .992$$

$$P(+ | \text{cancer}) = .98, P(- | \text{cancer}) = .02$$

$$P(+ | \neg \text{cancer}) = .03, P(- | \neg \text{cancer}) = .97$$

$$P(\text{cancer} | +) = \frac{P(+ | \text{cancer})P(\text{cancer})}{P(+)}$$

$$P(\neg \text{cancer} | +) = \frac{P(+ | \neg \text{cancer})P(\neg \text{cancer})}{P(+)}$$

### Example - Does patient have cancer or not?

- The test returns a correct positive result in only 98% of the cases in which the disease is actually present, and a correct negative result in only 97% of the cases in which the disease is not present.
- Furthermore, .008 of the entire population have cancer.

$$P(\text{cancer}) = .008$$

$$P(\text{notcancer}) = .992$$

$$P(+ | \text{cancer}) = .98$$

$$P(- | \text{cancer}) = .02$$

$$P(+ | \text{notcancer}) = .03$$

$$P(- | \text{notcancer}) = .97$$

- A patient takes a lab test and the result comes back positive.

$$P(+ | \text{cancer}) P(\text{cancer}) = .98 * .008 = .0078$$

$$P(+ | \text{notcancer}) P(\text{notcancer}) = .03 * .992 = .0298$$

→  $h_{MAP}$  is notcancer

- Since  $P(\text{cancer}|+) + P(\text{notcancer}|+)$  must be 1

$$P(\text{cancer}|+) = .0078 / (.0078 + .0298) = .21$$

$$P(\text{notcancer}|+) = .0298 / (.0078 + .0298) = .79$$

---

## MAP Learner (Maximum A Posteriori)

For each hypothesis  $h$  in  $H$ , calculate the posterior probability

$$P(h | D) = \frac{P(D | h)P(h)}{P(D)}$$

Output the hypothesis  $h_{\text{map}}$  with the highest posterior probability

$$h_{\text{map}} = \max_{h \in H} P(h | D)$$

Comments:

Computational intensive

Providing a standard for judging the performance of learning algorithms

Choosing  $P(h)$  and  $P(D|h)$  reflects our prior knowledge about the learning task

Naïve Bayesian Classification:

- Naïve assumption: attribute independence
- $P(x_1, \dots, x_k | C) = P(x_1 | C) \cdot \dots \cdot P(x_k | C)$
- If  $i$ -th attribute is categorical:
  - $P(x_i | C)$  is estimated as the relative freq of samples having value  $x_i$  as  $i$ -th attribute in class  $C$
- If  $i$ -th attribute is continuous:
  - $P(x_i | C)$  is estimated thru a Gaussian density function
- Computationally easy in both cases

### What is Naive Bayes algorithm?

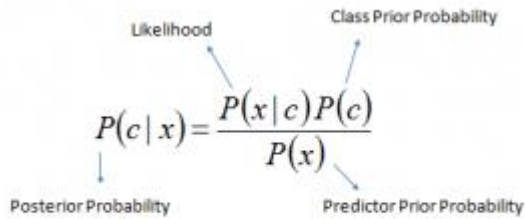
It is a classification technique based on Bayes' Theorem with an assumption of independence among predictors. In simple terms, a Naive Bayes classifier assumes that the presence of a particular feature in a class is unrelated to the presence of any other feature.

For example, a fruit may be considered to be an apple if it is red, round, and about 3 inches in diameter. Even if these features depend on each other or upon the existence of the other features, all of these properties independently contribute to the probability that this fruit is an apple and that is why it is known as 'Naive'.

Naive Bayes model is easy to build and particularly useful for very large data sets. Along with simplicity, Naive Bayes is known to outperform even highly sophisticated classification methods.

Bayes theorem provides a way of calculating posterior probability  $P(c|x)$  from  $P(c)$ ,  $P(x)$  and  $P(x|c)$ . Look at the equation below:

---



$$P(c|x) = \frac{P(x|c)P(c)}{P(x)}$$

Labels in diagram:  
 Likelihood:  $P(x|c)$   
 Class Prior Probability:  $P(c)$   
 Posterior Probability:  $P(c|x)$   
 Predictor Prior Probability:  $P(x)$

$$P(c|X) = P(x_1|c) \times P(x_2|c) \times \dots \times P(x_n|c) \times P(c)$$

Above,

- ☐  $P(c|x)$  is the posterior probability of class (c, target) given predictor (x, attributes).
- ☐  $P(c)$  is the prior probability of class.
- ☐  $P(x|c)$  is the likelihood which is the probability of predictor given class.
- ☐  $P(x)$  is the prior probability of predictor.

### How Naive Bayes algorithm works?



| Weather  | Play |
|----------|------|
| Sunny    | No   |
| Overcast | Yes  |
| Rainy    | Yes  |
| Sunny    | Yes  |
| Sunny    | Yes  |
| Overcast | Yes  |
| Rainy    | No   |
| Rainy    | No   |
| Sunny    | Yes  |
| Rainy    | Yes  |
| Sunny    | No   |
| Overcast | Yes  |
| Overcast | Yes  |
| Rainy    | No   |

| Frequency Table |    |     |
|-----------------|----|-----|
| Weather         | No | Yes |
| Overcast        |    | 4   |
| Rainy           | 3  | 2   |
| Sunny           | 2  | 3   |
| Grand Total     | 5  | 9   |

| Likelihood table |       |       |
|------------------|-------|-------|
| Weather          | No    | Yes   |
| Overcast         |       | 4     |
| Rainy            | 3     | 2     |
| Sunny            | 2     | 3     |
| All              | 5     | 9     |
|                  | =5/14 | =9/14 |
|                  | 0.36  | 0.64  |

Let's understand it using an example. Below I have a training data set of weather and corresponding target variable 'Play' (suggesting possibilities of playing). Now, we need to classify whether players will play or not based on weather condition. Let's follow the below steps to perform it.

Step 1: Convert the data set into a frequency table

Step 2: Create Likelihood table by finding the probabilities like Overcast probability = 0.29 and probability of playing is 0.64.

Step 3: Now, use Naive Bayesian equation to calculate the posterior probability for each class. The class with the highest posterior probability is the outcome of prediction.

**Problem:** Players will play if weather is sunny. Is this statement is correct? We can solve it using above discussed method of posterior probability.

$$P(\text{Yes} \mid \text{Sunny}) = P(\text{Sunny} \mid \text{Yes}) * P(\text{Yes}) / P(\text{Sunny})$$

Here we have  $P(\text{Sunny} \mid \text{Yes}) = 3/9 = 0.33$ ,  $P(\text{Sunny}) = 5/14 = 0.36$ ,  
 $P(\text{Yes}) = 9/14 = 0.64$  Now,  $P(\text{Yes} \mid \text{Sunny}) = 0.33 * 0.64 / 0.36 = 0.60$ , which has higher probability.

Naive Bayes uses a similar method to predict the probability of different class based on various attributes. This algorithm is mostly used in text classification and with problems having multiple classes.

---

## What are the Pros and Cons of Naive Bayes? Pros:

- It is easy and fast to predict class of test data set. It also perform well in multi class prediction
- When assumption of independence holds, a Naive Bayes classifier performs better compare to other models like logistic regression and you need less training data.
- It perform well in case of categorical input variables compared to numerical variable(s). For numerical variable, normal distribution is assumed (bell curve, which is a strong assumption).

## Cons:

- If categorical variable has a category (in test data set), which was not observed in training data set, then model will assign a 0 (zero) probability and will be unable to make a prediction. This is often known as “Zero Frequency”. To solve this, we can use the smoothing technique. One of the simplest smoothing techniques is called Laplace estimation.
- On the other side naive Bayes is also known as a bad estimator, so the probability outputs from predict\_proba are not to be taken too seriously.
- Another limitation of Naive Bayes is the assumption of independent predictors. In real life, it is almost impossible that we get a set of predictors which are completely independent.

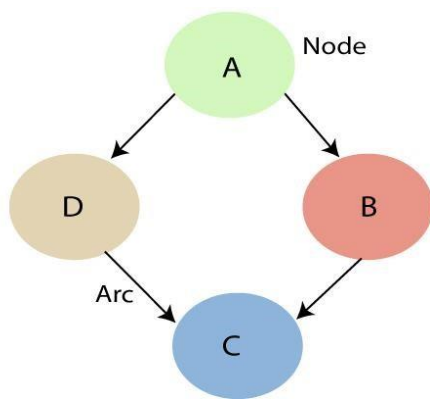
## Naive Bayes Algorithms Application

- **Real time Prediction:** Naive Bayes is an eager learning classifier and it is sure fast. Thus, it could be used for making predictions in real time.
- **Multi class Prediction:** This algorithm is also well known for multi class prediction feature. Here we can predict the probability of multiple classes of target variable.
- **Text classification/ Spam Filtering/ Sentiment Analysis:** Naive Bayes classifiers mostly used in text classification (due to better result in multi class problems and independence rule) have higher success rate as compared to other algorithms. As a result, it is widely used in Spam filtering (identify spam e-mail) and Sentiment Analysis (in social media analysis, to identify positive and negative customer sentiments)
- **Recommendation System:** Naive Bayes Classifier and Collaborative Filtering together builds a Recommendation System that uses machine learning and data mining techniques to filter unseen information and predict whether a user would like a given resource or not.



## What are Bayesian Networks?

By definition, Bayesian Networks are a type of Probabilistic Graphical Model that uses the Bayesian inferences for probability computations. It represents a set of variables and its conditional probabilities with a Directed Acyclic Graph (DAG). They are primarily suited for considering an event that has occurred and predicting the likelihood that any one of the several possible known causes is the contributing factor.



In the above diagram A, B, C and D are 4 random variables represented by nodes given in the network of the graph. To node B, A is its parent node and C is its child node. Node C is independent of Node A.

Before we get into the implementation of a Bayesian Network, there are a few probability basics that have to be understood.

### Conditional Probability

In mathematics, the Conditional Probability of event A is the probability that event A will occur given that another event B has already occurred. In simple terms,  $p(A | B)$  is the probability of event A occurring, given that event, B occurs. However, there are two types of event possibilities between A and B. They may be either dependent events or independent events. Depending upon their type, there are two different ways to calculate the conditional probability.

- Given A and B are dependent events, the conditional probability is calculated as  $P(A|B) = P(A \text{ and } B) / P(B)$
- If A and B are independent events, then the expression for conditional probability is given by,  $P(A|B) = P(A)$

## Joint Probability Distribution

Before we get into an example of Bayesian Networks, let us understand the concept of Joint Probability Distribution. Consider 3 variables  $a_1$ ,  $a_2$  and  $a_3$ . By definition, the probabilities of all different possible combinations of  $a_1$ ,  $a_2$ , and  $a_3$  are called its Joint Probability Distribution.

If  $P[a_1, a_2, a_3, \dots, a_n]$  is the JPD of the following variables from  $a_1$  to  $a_n$ , then there are several ways of calculating the Joint Probability Distribution as a combination of various terms such as,

$$P[a_1, a_2, a_3, \dots, a_n] = P[a_1 | a_2, a_3, \dots, a_n] * P[a_2, a_3, \dots, a_n]$$

$$= P[a_1 | a_2, a_3, \dots, a_n] * P[a_2 | a_3, \dots, a_n] \dots P[a_{n-1} | a_n] * P[a_n]$$

Generalizing the above equation, we can write the Joint Probability Distribution as,

$$P(X_i | X_{i-1}, \dots, X_n) = P(X_i | \text{Parents}(X_i))$$

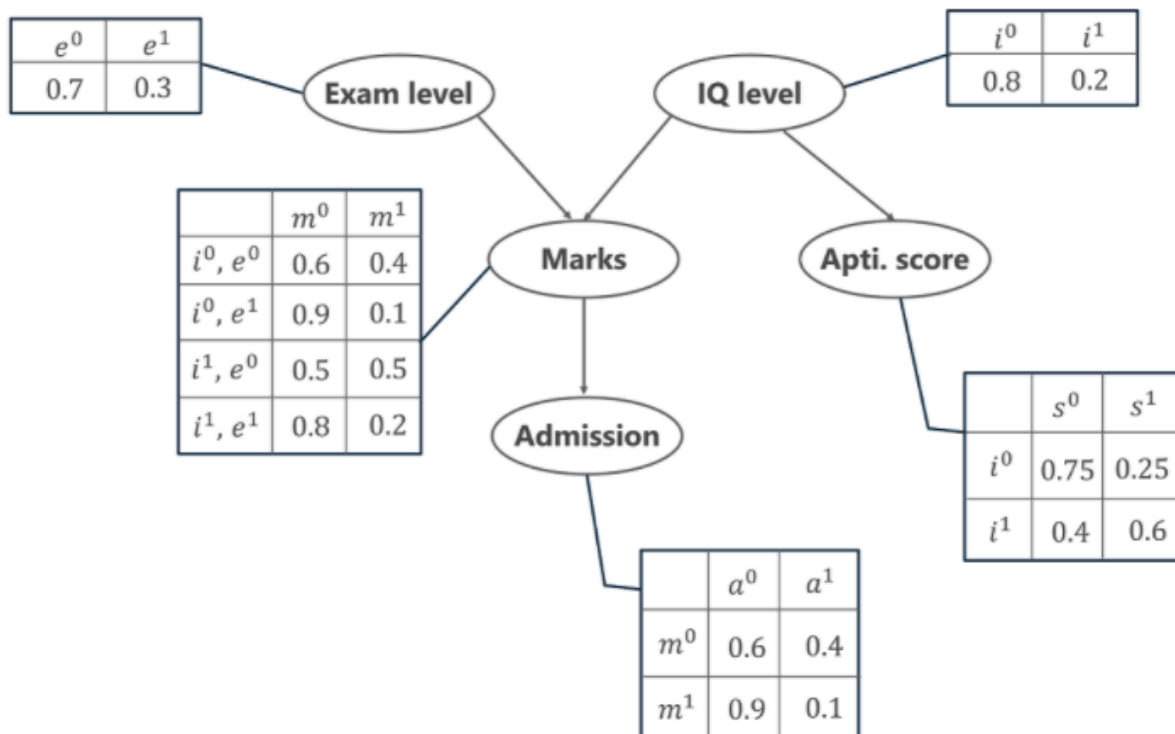
## Example of Bayesian Networks

Let us now understand the mechanism of Bayesian Networks and their advantages with the help of a simple example. In this example, let us imagine that we are given the task of modeling a student's marks ( $m$ ) for an exam he has just given. From the given Bayesian Network Graph below, we see that the marks depend upon two other variables. They are,

- Exam Level ( $e$ )– This discrete variable denotes the difficulty of the exam and has two values (0 for easy and 1 for difficult)

- IQ Level ( $i$ ) – This represents the Intelligence Quotient level of the student and is also discrete in nature having two values (0 for low and 1 for high)

Additionally, the IQ level of the student also leads us to another variable, which is the Aptitude Score of the student ( $s$ ). Now, with marks the student has scored, he can secure admission to a particular university. The probability distribution for getting admitted ( $a$ ) to a university is also given below.



In the above graph, we see several tables representing the probability distribution values of the given 5 variables. These tables are called the Conditional Probabilities Table or CPT. There are a few properties of the CPT given below –

- The sum of the CPT values in each row must be equal to 1 because all the possible cases for a particular variable are exhaustive (representing all possibilities).
- If a variable that is Boolean in nature has  $k$  Boolean parents, then in the CPT it has  $2^k$  probability values.

Coming back to our problem, let us first list all the possible events that are occurring in the above-given table.

1. Exam Level (e)
2. IQ Level (i)
3. Aptitude Score (s)
4. Marks (m)
5. Admission (a)

These five variables are represented in the form of a Directed Acyclic Graph (DAG) in a Bayesian Network format with their Conditional Probability tables. Now, to calculate the Joint Probability Distribution of the 5 variables the formula is given by,

$$P[a, m, i, e, s] = P(a | m) \cdot P(m | i, e) \cdot P(i) \cdot P(e) \cdot P(s | i)$$

From the above formula,

- $P(a | m)$  denotes the conditional probability of the student getting admission based on the marks he has scored in the examination.
- $P(m | i, e)$  represents the marks that the student will score given his IQ level and difficulty of the Exam Level.
- $P(i)$  and  $P(e)$  represent the probability of the IQ Level and the Exam Level.
- $P(s | i)$  is the conditional probability of the student's Aptitude Score, given his IQ Level.

With the following probabilities calculated, we can find the Joint Probability Distribution of the entire Bayesian Network.

### Calculation of Joint Probability Distribution

Let us now calculate the JPD for two cases.

**Case 1:** Calculate the probability that in spite of the exam level being difficult, the student having a low IQ level and a low Aptitude Score, manages to pass the exam and secure admission to the university.

From the above word problem statement, the Joint Probability Distribution can be written as below,

$$P[a=1, m=1, i=0, e=1, s=0]$$

From the above Conditional Probability tables, the values for the given conditions are fed to the formula and is calculated as below.

$$P[a=1, m=1, i=0, e=0, s=0] = P(a=1 \mid m=1) \cdot P(m=1 \mid i=0, e=1) \cdot P(i=0) \cdot P(e=1) \cdot P(s=0 \mid i=0)$$

$$= 0.1 * 0.1 * 0.8 * 0.3 * 0.75$$

$$= \mathbf{0.0018}$$



## What is ANN?

An artificial neuron network (ANN) is a computational model based on the structure and functionality resembles biological neural networks.

Information that flows through the network affects the structure of the ANN because a neural network changes - or learns, in a sense - based on that input and output.

ANNs are considered nonlinear statistical data modeling tools where the complex relationships between inputs and outputs are modeled or patterns are found.

ANN is also known as a neural network.

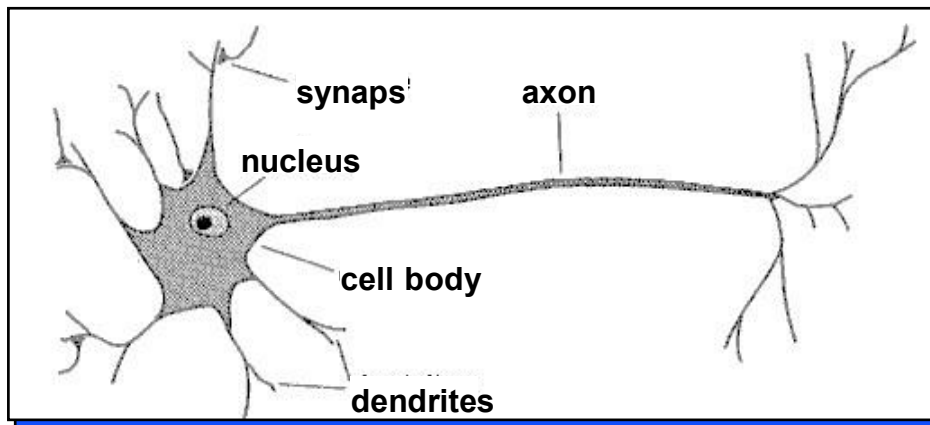
## History:

1943-McCulloch & Pitts are generally recognised as the designers of the first neural network  
1949-First learning rule

1969-Minsky & Papert - perceptron limitation - Death of ANN

1980's - Re-emergence of ANN - multi-layer networks

## The Structure of Neurons:

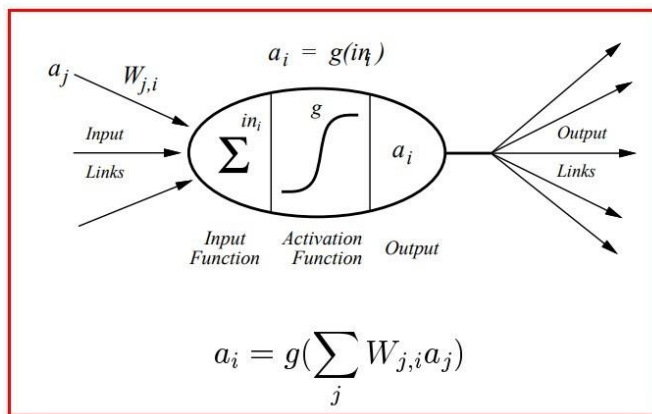


- A neuron only fires if its input signal exceeds a certain amount (the threshold) in a short time period.
- Synapses vary in strength

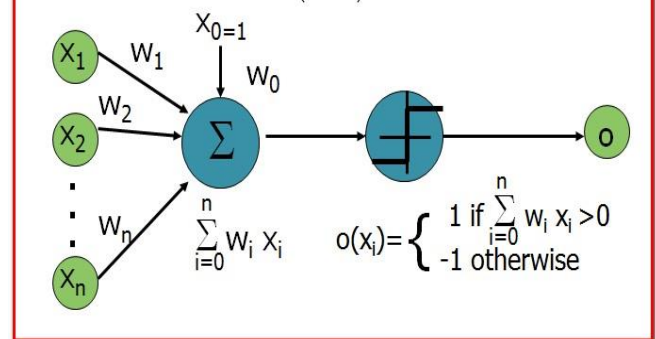
Synapses can be either excitatory or inhibitory.

- Many simple neuron-like threshold switching units
- Many weighted interconnections among units
- Highly parallel, distributed processing
- Learning by tuning the connection weights

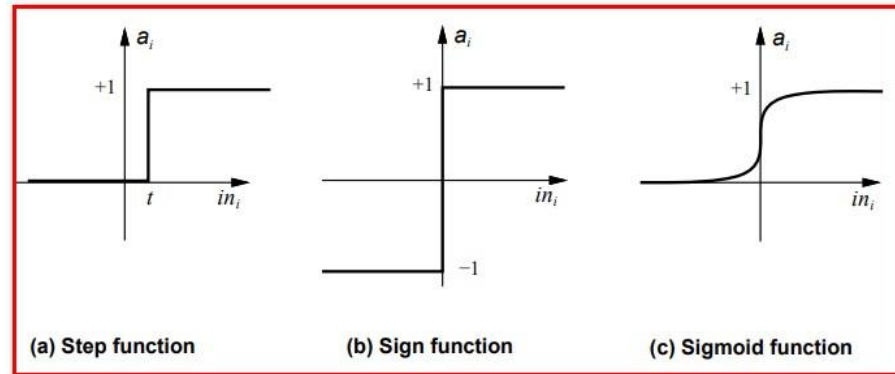
### Perceptron Main Processing Unit



### Linear threshold unit (LTU)



### Different Threshold Functions:



### Perceptron Learning Rule:

$$w_i \leftarrow w_i + \Delta w_i$$

$$\Delta w_i = \eta(t - o)x_i$$

$t=c(x)$  is

the

target

value  $o$

is the

perceptr

on

output

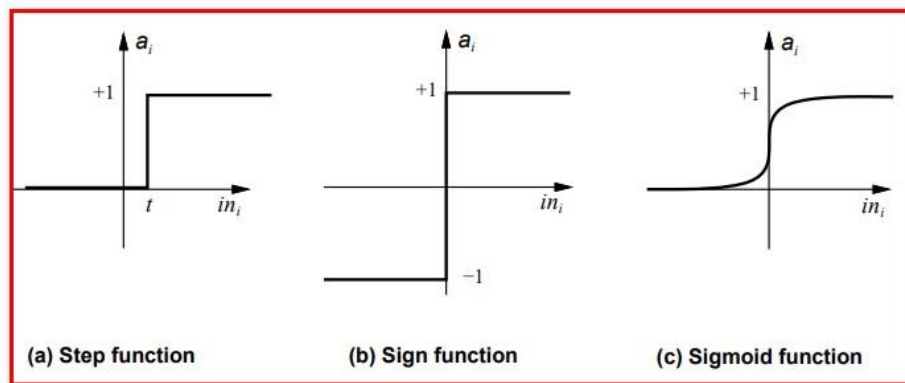
**Gamma is a small constant (e.g. 0.1) called *learning rate***

- If the output is correct ( $t=o$ , i.e., target = output) then weights  $w_i$  won't change.
- If the output is incorrect the weights  $w_i$  are changed such that the output of the perceptron for the new weights is *closer* to  $t$ .
- The algorithm converges to the correct classification if the training data is linearly separable and learning rate is sufficiently small

### Point of view of Perceptron:

- Developed by Frank Rosenblatt by using McCulloch and Pitts model, perceptron is the basic operational unit of artificial neural networks.
- Operational characteristics of the perceptron: It consists of a single neuron with an arbitrary number of inputs along with adjustable weights, but the output of the neuron is 1 or 0 depending upon the threshold.
- It also consists of a bias whose weight is always 1. Following figure gives a schematic representation of the perceptron.
- Perceptron thus has the three basic elements – **Links**, **Adder** and **Activation function**

### Sigmoid Function and Threshold unit:



- Threshold has a great impact on the topology of your neural network, based on the threshold neurons get on or off which means that if the value of activation function of a neuron exceeds the threshold, it outputs a quantity otherwise it will be off.

### Unit step (threshold)

- The output is set at one of two levels, depending on whether the total input is greater than or less than some threshold value.

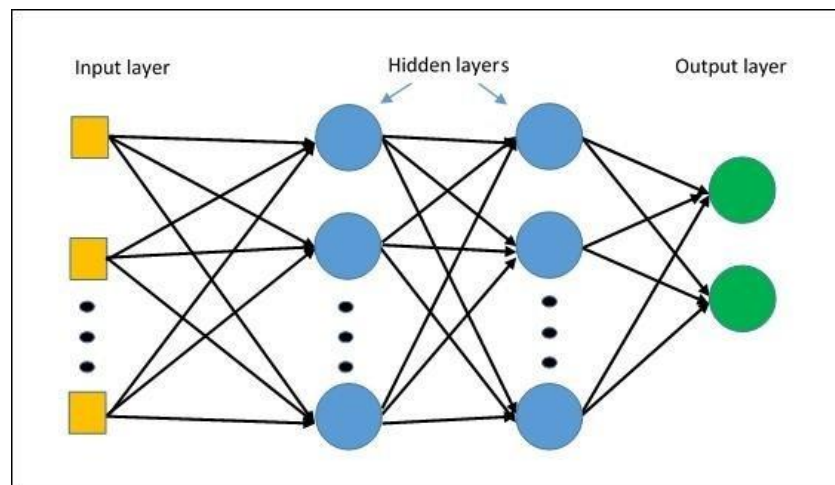
### Sigmoid

- The sigmoid function consists of 2 functions, logistic and tangential. The values of logistic function range from 0 and 1 and -1 to +1 for tangential function.

## Multilayer feed forward Network:

- A multilayer perceptron (MLP) is a class of feedforward artificial neural network. An MLP consists of at least three layers of nodes: an input layer, a hidden layer and an output layer.
- Except for the input nodes, each node is a neuron that uses a nonlinear activation function.
- MLP utilizes a supervised learning technique called backpropagation for training. Its multiple layers and non-linear activation distinguish MLP from a linear perceptron.

- It can



distinguish

data that is not linearly separable.

- Multilayer perceptrons are sometimes colloquially referred to as "vanilla" neural networks, especially when they have a single hidden layer.
- The term "multilayer perceptron" does not refer to a single perceptron that has multiple layers. Rather, it contains many perceptrons that are organized into layers. An alternative is "multilayer perceptron network".
- Moreover, MLP "perceptrons" are not perceptrons in the strictest possible sense. True perceptrons are formally a special case of artificial neurons that use a threshold activation function such as the Heaviside step function.

- MLP perceptrons can employ arbitrary activation functions. A true perceptron performs binary classification (either this or that), an MLP neuron is free to either perform classification or regression, depending upon its activation function.
- The term "multilayer perceptron" later was applied without respect to nature of the nodes/layers, which can be composed of arbitrarily defined artificial neurons, and not perceptrons specifically. This interpretation avoids the loosening of the definition of "perceptron" to mean an artificial neuron in general.

### Delta Rule:

The delta training rule is best understood by considering the task of training an un-thresholded perceptron; that is, a linear unit for which the output  $o$  is given by .

$$o(\vec{x}) = \vec{w} \cdot \vec{x}$$

Thus, a linear unit corresponds to the first stage of a perceptron, without the threshold.

Although there are many ways to define this error, one common measure that will turn out to be especially

convenient  $\frac{1}{2}$  —  
 $\frac{1}{2}$  is:

$$E[\vec{w}] \equiv \frac{1}{2} \sum_{d \in D} (t_d - o_d)^2$$

### Learning rate:

The size of gradient steps is called the *learning rate*. With a high learning rate we can cover more ground each step, but we risk overshooting the lowest point since the slope of the hill is constantly changing. With a very low learning rate, we can confidently move in the direction of the negative gradient since we are recalculating it so frequently. A low learning rate is more precise, but calculating the gradient is time-consuming, so it will take us a very long time to get to the bottom.

### Cost function

A [Loss Functions](#) tells us “how good” our model is at making predictions for a given set of parameters. The cost function has its own curve and its own gradients. The slope of this curve tells us how to update our parameters to make the model more accurate.

### Step-by-step

Now let's run gradient descent using our new cost function. There are two parameters in our cost function we can control:  $m$  (weight) and  $b$  (bias). Since we need to consider the impact each one has on the final prediction, we need to use partial derivatives. We calculate the partial derivatives of the cost function with respect to each parameter and store the results in a gradient. Math:

Given the cost function:

$$f(m, b) = \frac{1}{N} \sum_{i=1}^n (y_i - (mx_i + b))^2$$

The gradient can be calculated as:

$$f'(m, b) = \begin{bmatrix} \frac{df}{dm} \\ \frac{df}{db} \end{bmatrix} = \begin{bmatrix} \frac{1}{N} \sum -2x_i(y_i - (mx_i + b)) \\ \frac{1}{N} \sum -2(y_i - (mx_i + b)) \end{bmatrix}$$

To solve for the gradient, we iterate through our data points using our new  $m$  and  $b$  values and compute the partial derivatives. This new gradient tells us the slope of our cost function at our current position (current parameter values) and the direction we should move to update our parameters. The size of our update is controlled by the learning rate.

## Multilayer Artificial Neural Network:

A fully connected multi-layer neural network is called a Multilayer Perceptron (MLP).

Perceptron rule and Adaline rule were used to train a single-layer neural network. It has 3 layers including one hidden layer. If it has more than 1 hidden layer, it is called a deep ANN. An MLP is a typical example of a feedforward artificial neural network. In this figure, the  $i$ th activation unit in the  $l$ th layer is denoted as  $a_i(l)$ .

The number of layers and the number of neurons are referred to as hyperparameters of a neural network, and these need tuning. Cross-validation techniques must be used to find ideal values for these. The weight adjustment training is done via backpropagation. Deeper neural networks are better at processing data. However, deeper layers can lead to vanishing gradient problem. Special algorithms are required to solve this issue.

**Back-Propagation Algorithm working scenario:** This is a quite surprising fact, because it states that we know the form of the distribution that governs the sample mean ? even when we do not know the form of the underlying distribution that governs the individual  $Y_i$  that are being observed! Furthermore, the Central Limit Theorem describes how the mean and variance of  $Y$  can be used to determine the mean and variance of the individual  $Y_i$ . Back Propagation neural network

Multilayer neural networks use a most common technique from a variety of learning technique, called the back- propagation algorithm. In back propagation neural network, the output values are compared with the correct answer to compute the value of some predefined error function. By various techniques the error is then fed back through the network. Using this information, the algorithms adjust the weights of each connection in order to reduce the value of the error function by some small amount. After repeating this process for a sufficiently large number of training cycles the network will usually converge to some state where the error of the calculation is small.

The goal of back propagation, as with most training algorithms, is to iteratively adjust the weights in the network to produce the desired output by minimizing the output error. The algorithm's goal is to solve credit assignment problem. Back propagation is a gradient-descent approach in that it uses the minimization of first-order derivatives to find an optimal solution. The standard back propagation algorithm is given below.

**Step1:** Build a network with the chosen number of input, hidden and output units.

**Step2:** Initialize all the weights to low random values.

**Step3:** Randomly, choose a single training pair.

**Step4:** Copy the input pattern to the input layer.

**Step5:** Cycle the network so that the activation from the inputs generates the activations in the hidden and output layers.

**Step6:** Calculate the error derivative between the output activation and the final output.

**Step7:** Apply the method of back propagation to the summed products of the weights and errors in the output layer in order to calculate the error in the hidden units.

**Step8:** Update the weights attached the each unit according to the error in that unit, the output from the unit below it and the learning parameters, until the error is sufficiently low.

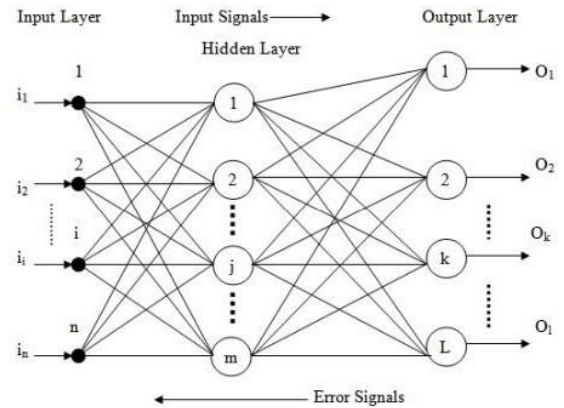


Figure Three layer back-propagation neural network



**Input:**

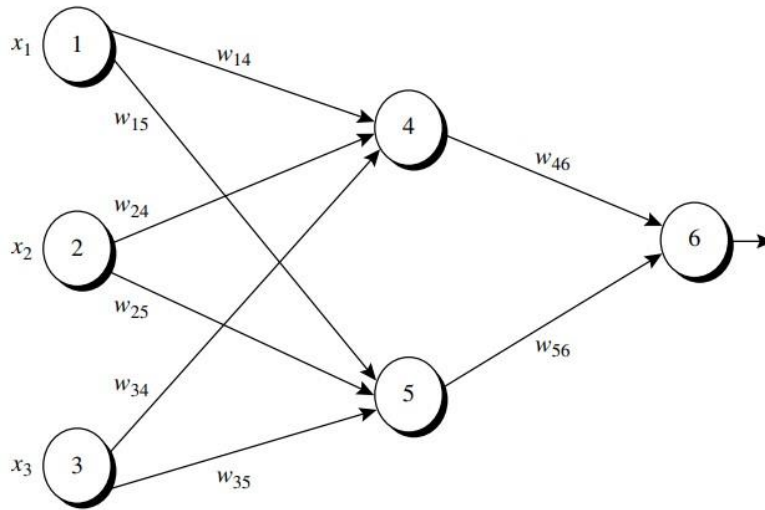
- $D$ , a data set consisting of the training tuples and their associated target values;
- $l$ , the learning rate;
- $network$ , a multilayer feed-forward network.

**Output:** A trained neural network.**Method:**

```

(1) Initialize all weights and biases in  $network$ ;
(2) while terminating condition is not satisfied {
(3)   for each training tuple  $X$  in  $D$  {
(4)     // Propagate the inputs forward:
(5)     for each input layer unit  $j$  {
(6)        $O_j = I_j$ ; // output of an input unit is its actual input value
(7)     for each hidden or output layer unit  $j$  {
(8)        $I_j = \sum_i w_{ij} O_i + \theta_j$ ; // compute the net input of unit  $j$  with respect to
        the previous layer,  $i$ 
(9)        $O_j = \frac{1}{1 + e^{-I_j}}$ ; } // compute the output of each unit  $j$ 
(10)    // Backpropagate the errors:
(11)    for each unit  $j$  in the output layer
(12)       $Err_j = O_j(1 - O_j)(T_j - O_j)$ ; // compute the error
(13)    for each unit  $j$  in the hidden layers, from the last to the first hidden layer
(14)       $Err_j = O_j(1 - O_j) \sum_k Err_k w_{jk}$ ; // compute the error with respect to
        the next higher layer,  $k$ 
(15)    for each weight  $w_{ij}$  in  $network$  {
(16)       $\Delta w_{ij} = (l) Err_j O_i$ ; // weight increment
(17)       $w_{ij} = w_{ij} + \Delta w_{ij}$ ; } // weight update
(18)    for each bias  $\theta_j$  in  $network$  {
(19)       $\Delta \theta_j = (l) Err_j$ ; // bias increment
(20)       $\theta_j = \theta_j + \Delta \theta_j$ ; } // bias update
(21)  } }
```

To derive the back propagation algorithm, let us consider the three layer network shown in figure.



**re 9.5** Example of a multilayer feed-forward neural network.

**le 9.1** Initial Input, Weight, and Bias Values

| $x_1$ | $x_2$ | $x_3$ | $w_{14}$ | $w_{15}$ | $w_{24}$ | $w_{25}$ | $w_{34}$ | $w_{35}$ | $w_{46}$ | $w_{56}$ | $\theta_4$ | $\theta_5$ | $\theta_6$ |
|-------|-------|-------|----------|----------|----------|----------|----------|----------|----------|----------|------------|------------|------------|
| 1     | 0     | 1     | 0.2      | -0.3     | 0.4      | 0.1      | -0.5     | 0.2      | -0.3     | -0.2     | -0.4       | 0.2        | 0.1        |

**Table 9.2** Net Input and Output Calculations

| <i>Unit, j</i> | <i>Net Input, I<sub>j</sub></i>               | <i>Output, O<sub>j</sub></i> |
|----------------|-----------------------------------------------|------------------------------|
| 4              | $0.2 + 0 - 0.5 - 0.4 = -0.7$                  | $1/(1 + e^{0.7}) = 0.332$    |
| 5              | $-0.3 + 0 + 0.2 + 0.2 = 0.1$                  | $1/(1 + e^{-0.1}) = 0.525$   |
| 6              | $(-0.3)(0.332) - (0.2)(0.525) + 0.1 = -0.105$ | $1/(1 + e^{0.105}) = 0.474$  |

**Table 9.3** Calculation of the Error at Each Node

| <i>Unit, j</i> | <i>Err<sub>j</sub></i>                       |
|----------------|----------------------------------------------|
| 6              | $(0.474)(1 - 0.474)(1 - 0.474) = 0.1311$     |
| 5              | $(0.525)(1 - 0.525)(0.1311)(-0.2) = -0.0065$ |
| 4              | $(0.332)(1 - 0.332)(0.1311)(-0.3) = -0.0087$ |

**Table 9.4** Calculations for Weight and Bias Updating

| <i>Weight<br/>or Bias</i> | <i>New Value</i>                       |
|---------------------------|----------------------------------------|
| $w_{46}$                  | $-0.3 + (0.9)(0.1311)(0.332) = -0.261$ |
| $w_{56}$                  | $-0.2 + (0.9)(0.1311)(0.525) = -0.138$ |
| $w_{14}$                  | $0.2 + (0.9)(-0.0087)(1) = 0.192$      |
| $w_{15}$                  | $-0.3 + (0.9)(-0.0065)(1) = -0.306$    |
| $w_{24}$                  | $0.4 + (0.9)(-0.0087)(0) = 0.4$        |
| $w_{25}$                  | $0.1 + (0.9)(-0.0065)(0) = 0.1$        |
| $w_{34}$                  | $-0.5 + (0.9)(-0.0087)(1) = -0.508$    |
| $w_{35}$                  | $0.2 + (0.9)(-0.0065)(1) = 0.194$      |
| $\theta_6$                | $0.1 + (0.9)(0.1311) = 0.218$          |
| $\theta_5$                | $0.2 + (0.9)(-0.0065) = 0.194$         |
| $\theta_4$                | $-0.4 + (0.9)(-0.0087) = -0.408$       |

## K-Nearest Neighbor

- K-Nearest Neighbor is one of the simplest Machine Learning algorithms based on Supervised Learning technique.
- K-NN algorithm assumes the similarity between the new case/data and available cases and put the new case into the category that is most similar to the available categories.
- K-NN algorithm stores all the available data and classifies a new data point based on the similarity. This means when new data appears then it can be easily classified into a well suite category by using K- NN algorithm.
- K-NN algorithm can be used for Regression as well as for Classification but mostly it is used for the Classification problems.
- K-NN is a non-parametric algorithm, which means it does not make any assumption on underlying data.
- It is also called a lazy learner algorithm because it does not learn from the training set immediately instead it stores the dataset and at the time of classification, it performs an action on the dataset.
- KNN algorithm at the training phase just stores the dataset and when it gets new data, then it classifies that data into a category that is much similar to the new data.

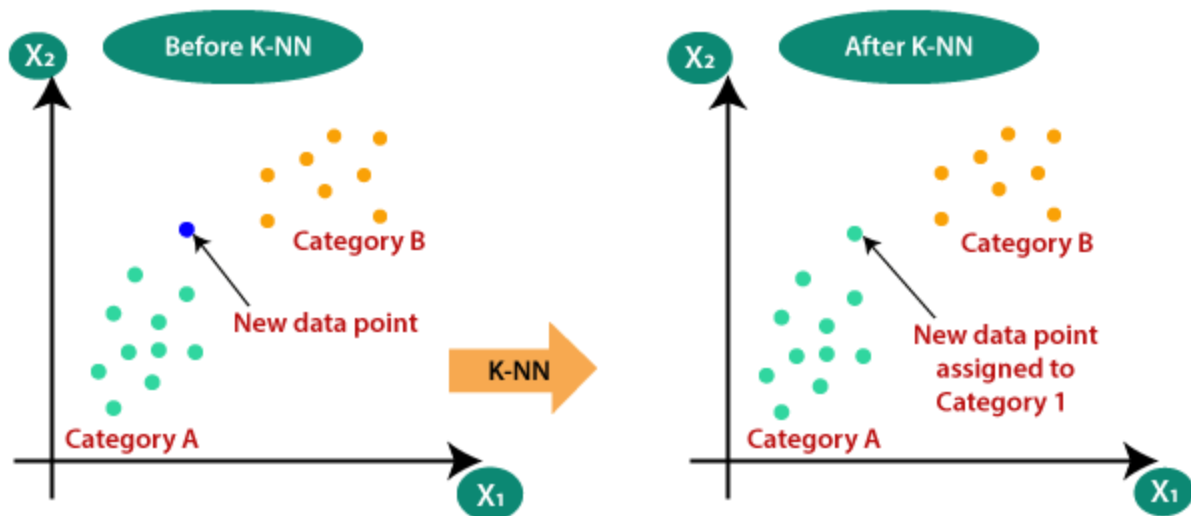
Example: Suppose, we have an image of a creature that looks similar to cat and dog, but we want to know either it is a cat or dog. So for this identification, we can use the KNN algorithm, as it works on a similarity measure. Our KNN model will find the similar features of the new data set to the cats and dogs images and based on the most similar features it will put it in either cat or



dog category.

### Need of KNN Algorithm

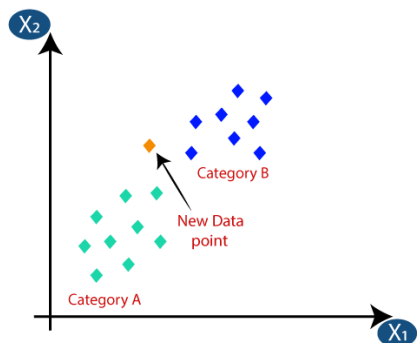
Suppose there are two categories, i.e., Category A and Category B, and we have a new data point  $x_1$ , so this data point will lie in which of these categories. To solve this type of problem, we need a K-NN algorithm. With the help of K-NN, we can easily identify the category or class of a particular dataset. Consider the below diagram:



The K-NN working can be explained on the basis of the below algorithm:

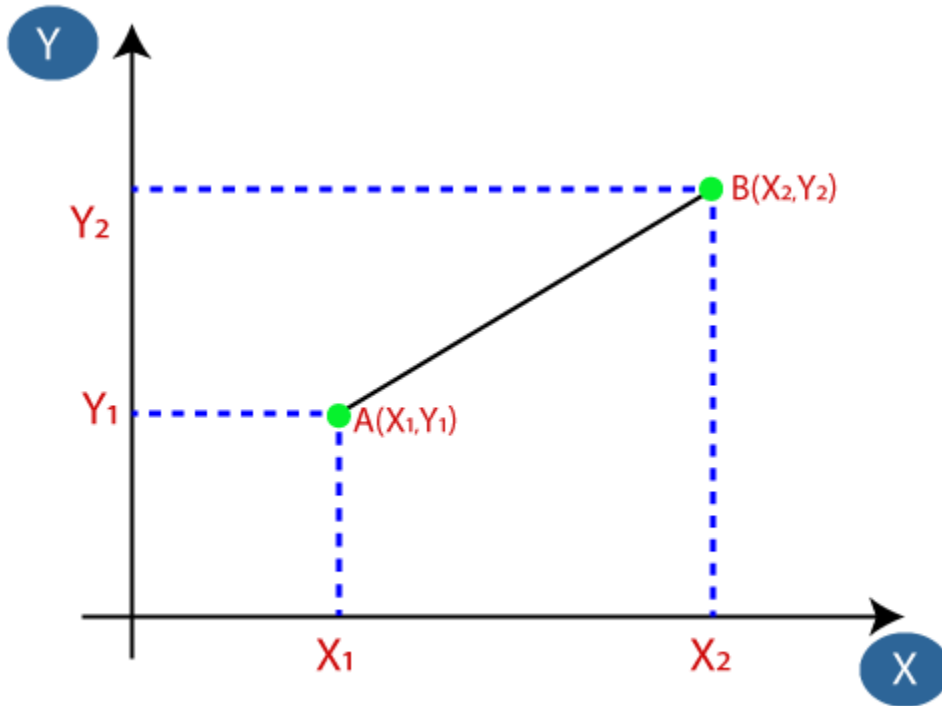
- **Step-1:** Select the number K of the neighbors
- **Step-2:** Calculate the Euclidean distance of **K number of neighbors**
- **Step-3:** Take the K nearest neighbors as per the calculated Euclidean distance.
- **Step-4:** Among these k neighbors, count the number of the data points in each category.
- **Step-5:** Assign the new data points to that category for which the number of the neighbor is maximum.
- **Step-6:** Our model is ready.

Suppose we have a new data point and we need to put it in the required category. Consider the below image:



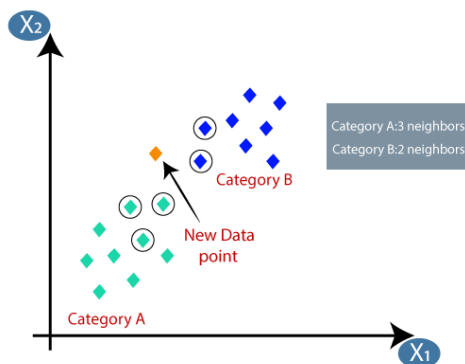
- Firstly, we will choose the number of neighbors, so we will choose the  $k=5$ .

- Next, we will calculate the **Euclidean distance** between the data points. The Euclidean distance is the distance between two points, which we have already studied in geometry. It can be calculated as:



$$\text{Euclidean Distance between } A_1 \text{ and } B_2 = \sqrt{(X_2 - X_1)^2 + (Y_2 - Y_1)^2}$$

- By calculating the Euclidean distance we got the nearest neighbors, as three nearest neighbors in category A and two nearest neighbors in category B. Consider the below image:



As we can see the 3 nearest neighbors are from category A, hence this new data point must belong to category A.

Below are some points to remember while selecting the value of K in the K-NN algorithm:

- There is no particular way to determine the best value for "K", so we need to try some values to find the best out of them. The most preferred value for K is 5.
- A very low value for K such as  $K=1$  or  $K=2$ , can be noisy and lead to the effects of outliers in the model.
- Large values for K are good, but it may find some difficulties.

### **Advantages**

- It is simple to implement.
- It is robust to the noisy training data
- It can be more effective if the training data is large.

### **Disadvantages**

- Always needs to determine the value of K which may be complex some time.
- The computation cost is high because of calculating the distance between the data points for all the training samples.

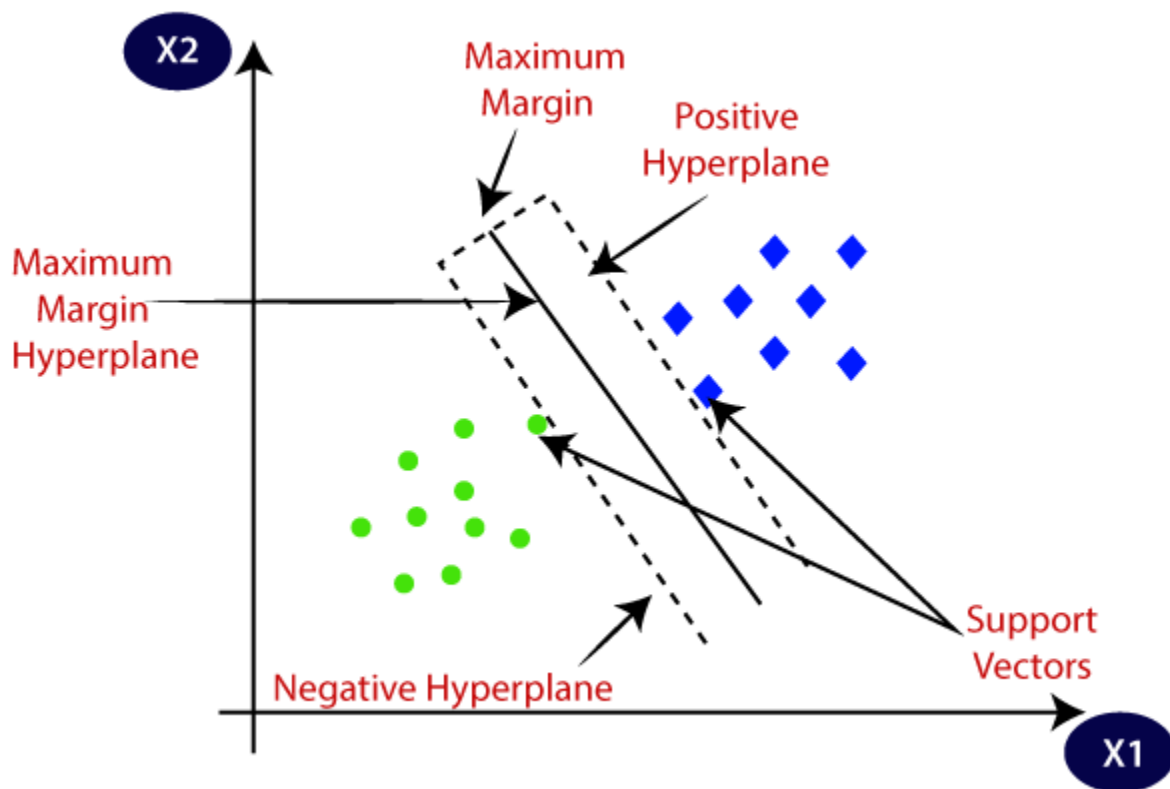


## Support Vector Machine

Support Vector Machine or SVM is one of the most popular Supervised Learning algorithms, which is used for Classification as well as Regression problems. However, primarily, it is used for Classification problems in Machine Learning.

The goal of the SVM algorithm is to create the best line or decision boundary that can segregate n-dimensional space into classes so that we can easily put the new data point in the correct category in the future. This best decision boundary is called a hyperplane.

SVM chooses the extreme points/vectors that help in creating the hyperplane. These extreme cases are called support vectors, and hence algorithm is termed as Support Vector Machine. Consider the below diagram in which there are two different categories that are classified using a decision boundary or hyperplane:



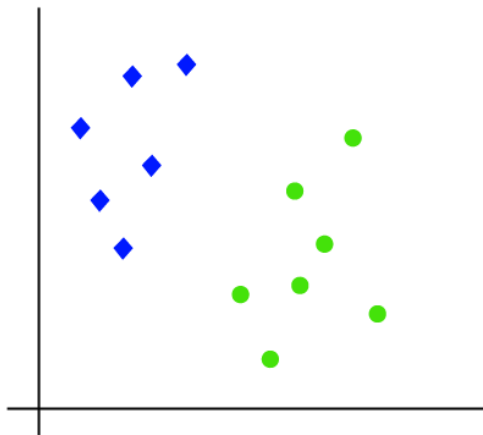
Types of SVM

**SVM can be of two types:**

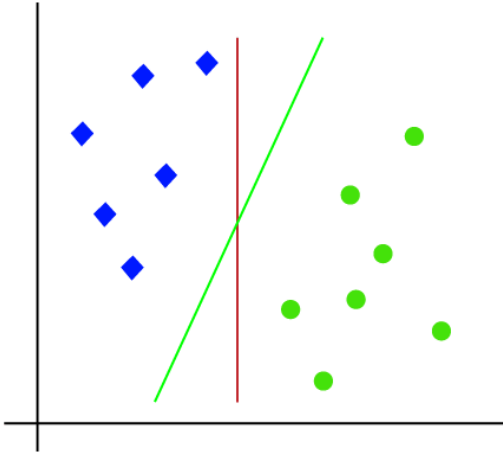
- **Linear SVM:** Linear SVM is used for linearly separable data, which means if a dataset can be classified into two classes by using a single straight line, then such data is termed as linearly separable data, and classifier is used called as Linear SVM classifier.
- **Non-linear SVM:** Non-Linear SVM is used for non-linearly separated data, which means if a dataset cannot be classified by using a straight line, then such data is termed as non-linear data and classifier used is called as Non-linear SVM classifier.
- **Hyperplane:** There can be multiple lines/decision boundaries to segregate the classes in n-dimensional space, but we need to find out the best decision boundary that helps to classify the data points. This best boundary is known as the hyperplane of SVM.
- The dimensions of the hyperplane depend on the features present in the dataset, which means if there are 2 features (as shown in image), then hyperplane will be a straight line. And if there are 3 features, then hyperplane will be a 2-dimension plane.
- We always create a hyperplane that has a maximum margin, which means the maximum distance between the data points.
- **Support Vectors:** The data points or vectors that are the closest to the hyperplane and which affect the position of the hyperplane are termed as Support Vector. Since these vectors support the hyperplane, hence called a Support vector.

### Linear SVM:

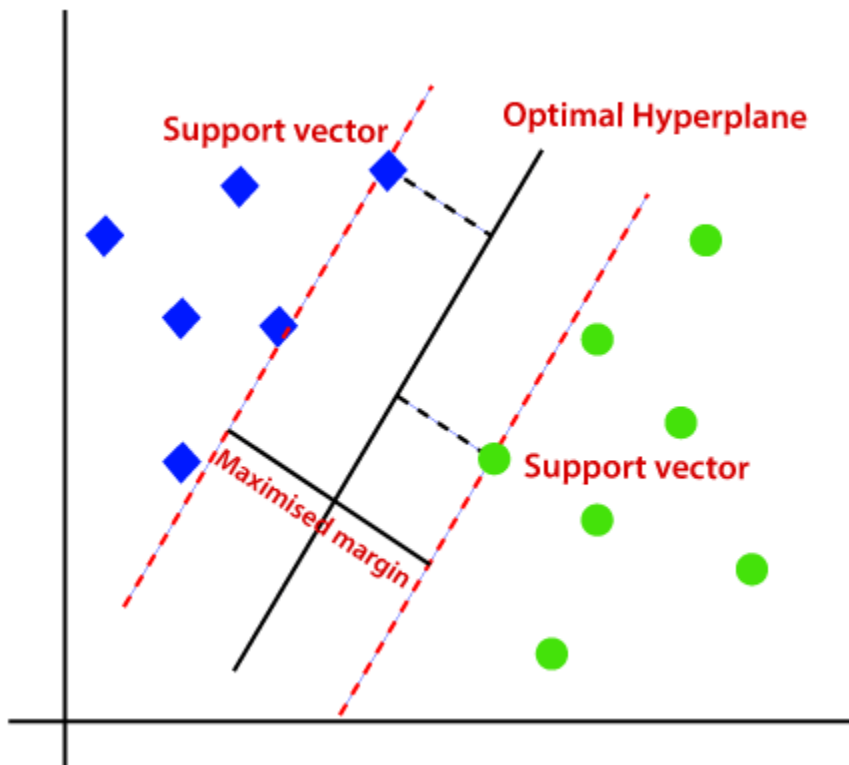
The working of the SVM algorithm can be understood by using an example. Suppose we have a dataset that has two tags (green and blue), and the dataset has two features  $x_1$  and  $x_2$ . We want a classifier that can classify the pair( $x_1$ ,  $x_2$ ) of coordinates in either green or blue. Consider the below image:



So as it is 2-d space so by just using a straight line, we can easily separate these two classes. But there can be multiple lines that can separate these classes. Consider the below image:

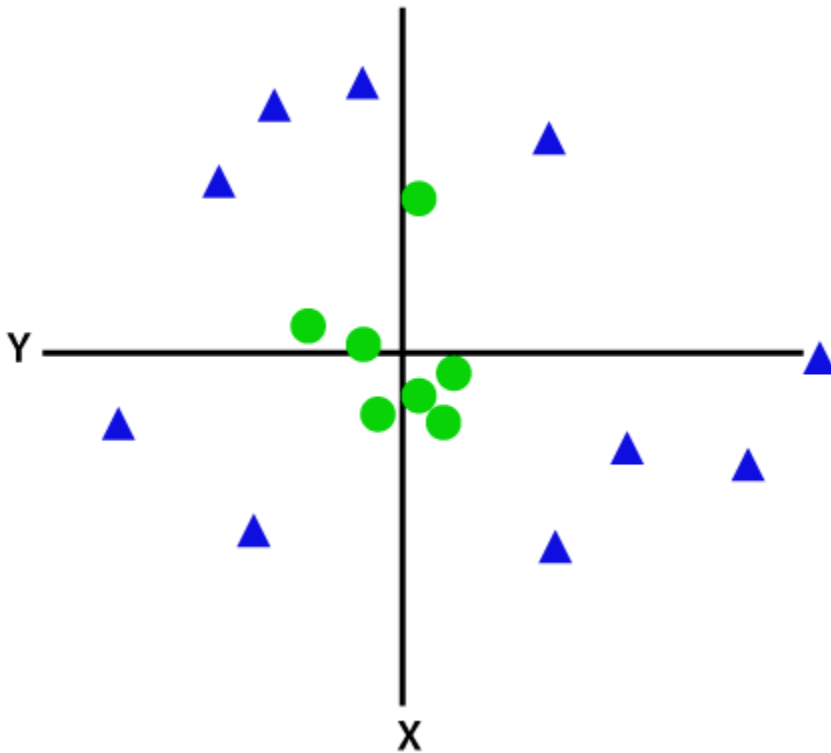


Hence, the SVM algorithm helps to find the best line or decision boundary; this best boundary or region is called as a **hyperplane**. SVM algorithm finds the closest point of the lines from both the classes. These points are called support vectors. The distance between the vectors and the hyperplane is called as **margin**. And the goal of SVM is to maximize this margin. The **hyperplane** with maximum margin is called the **optimal hyperplane**.



**Non-Linear SVM:**

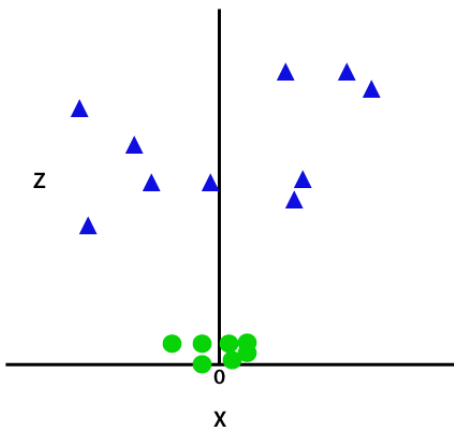
If data is linearly arranged, then we can separate it by using a straight line, but for non-linear data, we cannot draw a single straight line. Consider the below image:



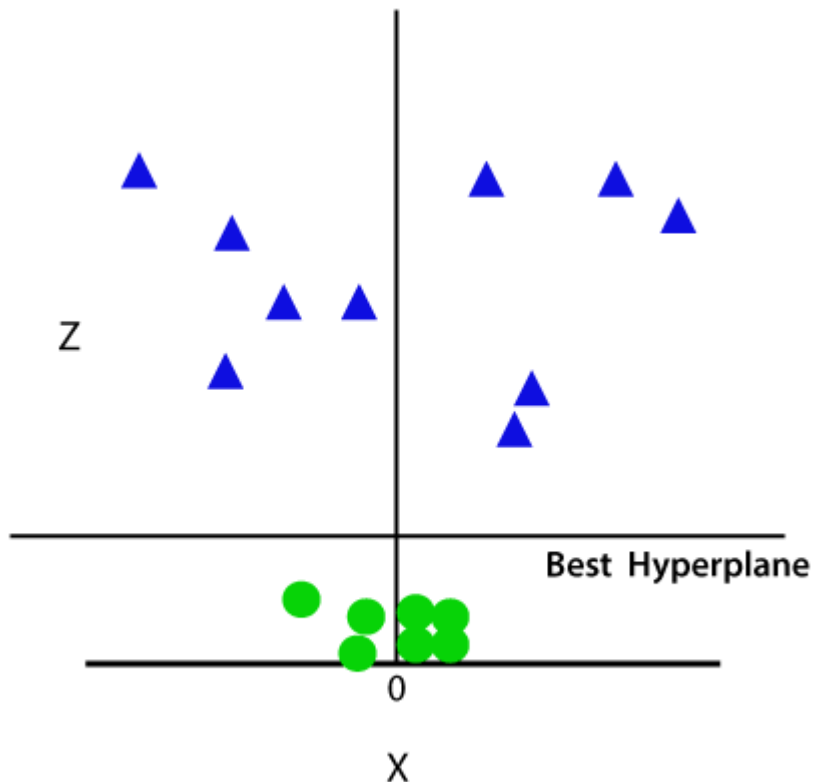
So to separate these data points, we need to add one more dimension. For linear data, we have used two dimensions x and y, so for non-linear data, we will add a third dimension z. It can be calculated as:

$$z = x^2 + y^2$$

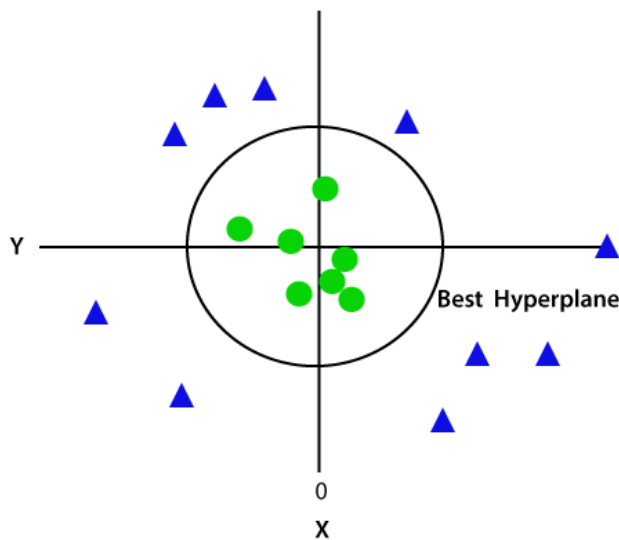
By adding the third dimension, the sample space will become as below image:



So now, SVM will divide the datasets into classes in the following way. Consider the below image:



Since we are in 3-d Space, hence it is looking like a plane parallel to the x-axis. If we convert it in 2d space with  $z=1$ , then it will become as:



Hence we get a circumference of radius 1 in case of non-linear data.

### **Advantages of SVM**

1. **Effective in High Dimensions:** Performs well in high-dimensional spaces.
2. **Robust to Overfitting:** Especially with the proper choice of kernel and regularization.
3. **Versatile:** Can be used for both linear and non-linear classification tasks.
4. **Memory Efficient:** Uses a subset of the training data (support vectors) for decision making.

### **Limitations of SVM**

1. **Computationally Intensive:** Training can be slow, especially for large datasets.
2. **Choice of Kernel:** Requires careful selection of the kernel function and tuning of parameters.
3. **Interpretability:** The resulting model is often less interpretable compared to simpler models like linear regression.

## What is ANN?

An artificial neuron network (ANN) is a computational model based on the structure and functionality resembles biological neural networks.

Information that flows through the network affects the structure of the ANN because a neural network changes - or learns, in a sense - based on that input and output.

ANNs are considered nonlinear statistical data modeling tools where the complex relationships between inputs and outputs are modeled or patterns are found.

ANN is also known as a neural network.

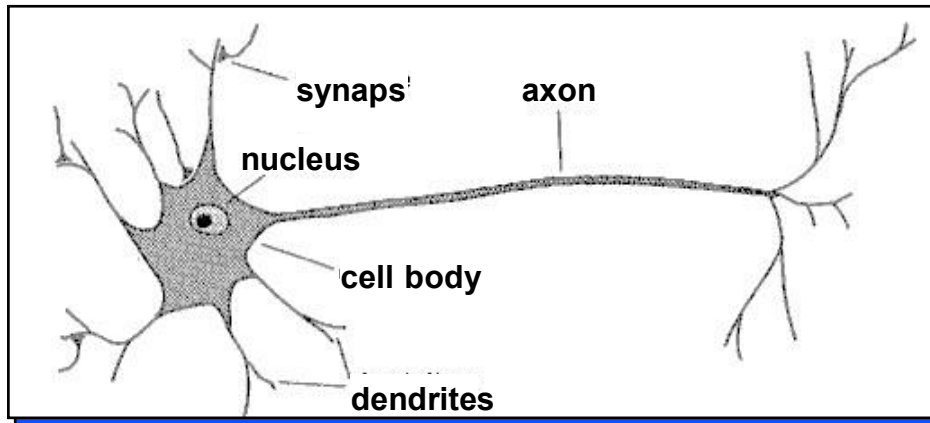
## History:

1943-McCulloch & Pitts are generally recognised as the designers of the first neural network  
1949-First learning rule

1969-Minsky & Papert - perceptron limitation - Death of ANN

1980's - Re-emergence of ANN - multi-layer networks

## The Structure of Neurons:

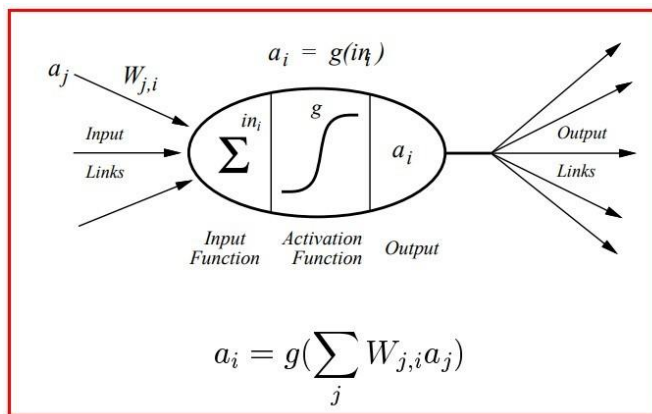


- A neuron only fires if its input signal exceeds a certain amount (the threshold) in a short time period.
- Synapses vary in strength

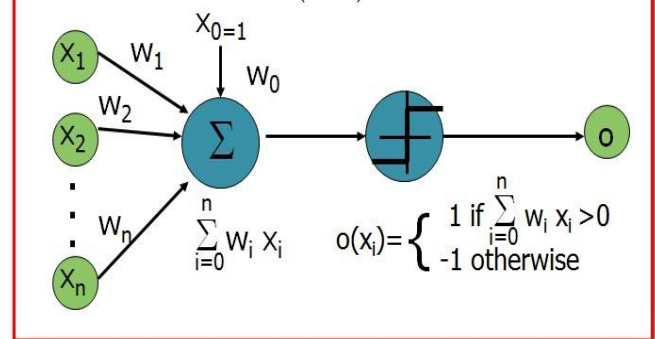
Synapses can be either excitatory or inhibitory.

- Many simple neuron-like threshold switching units
- Many weighted interconnections among units
- Highly parallel, distributed processing
- Learning by tuning the connection weights

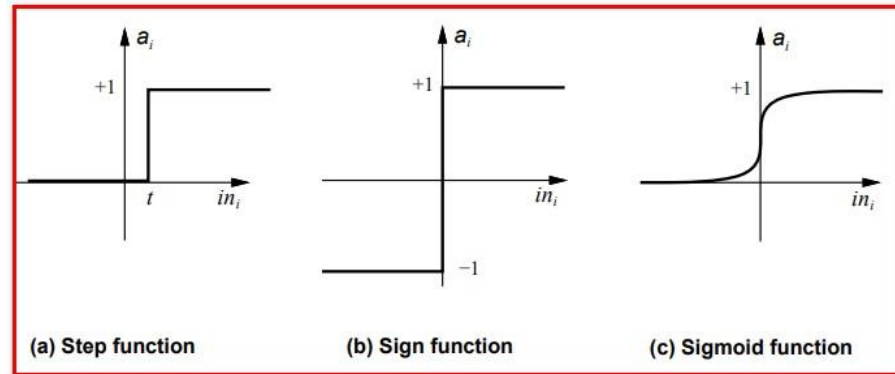
### Perceptron Main Processing Unit



### Linear threshold unit (LTU)



### Different Threshold Functions:



### Perceptron Learning Rule:

$$w_i \leftarrow w_i + \Delta w_i$$

$$\Delta w_i = \eta(t - o)x_i$$

$t=c(x)$  is

the



target

value  $o$

is the

perceptr

on

output

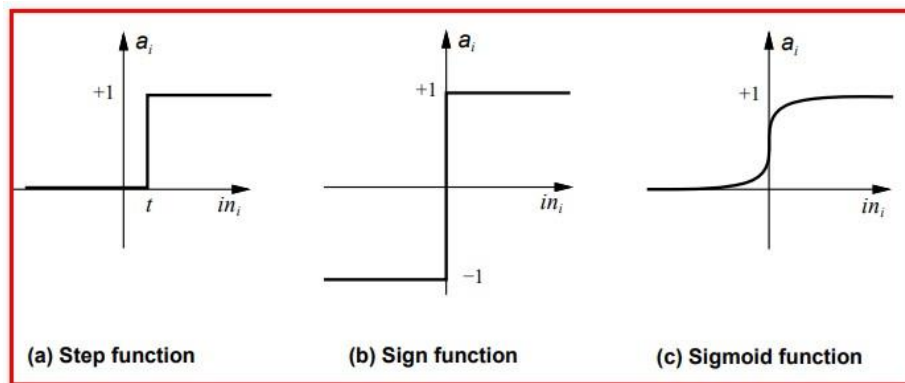
**Gamma is a small constant (e.g. 0.1) called *learning rate***

- If the output is correct ( $t=o$ , i.e., target = output) then weights  $w_i$  won't change.
- If the output is incorrect the weights  $w_i$  are changed such that the output of the perceptron for the new weights is *closer* to  $t$ .
- The algorithm converges to the correct classification if the training data is linearly separable and learning rate is sufficiently small

### Point of view of Perceptron:

- Developed by Frank Rosenblatt by using McCulloch and Pitts model, perceptron is the basic operational unit of artificial neural networks.
- Operational characteristics of the perceptron: It consists of a single neuron with an arbitrary number of inputs along with adjustable weights, but the output of the neuron is 1 or 0 depending upon the threshold.
- It also consists of a bias whose weight is always 1. Following figure gives a schematic representation of the perceptron.
- Perceptron thus has the three basic elements – **Links**, **Adder** and **Activation function**

### Sigmoid Function and Threshold unit:



- Threshold has a great impact on the topology of your neural network, based on the threshold neurons get on or off which means that if the value of activation function of a neuron exceeds the threshold, it outputs a quantity otherwise it will be off.

### Unit step (threshold)

- The output is set at one of two levels, depending on whether the total input is greater than or less than some threshold value.

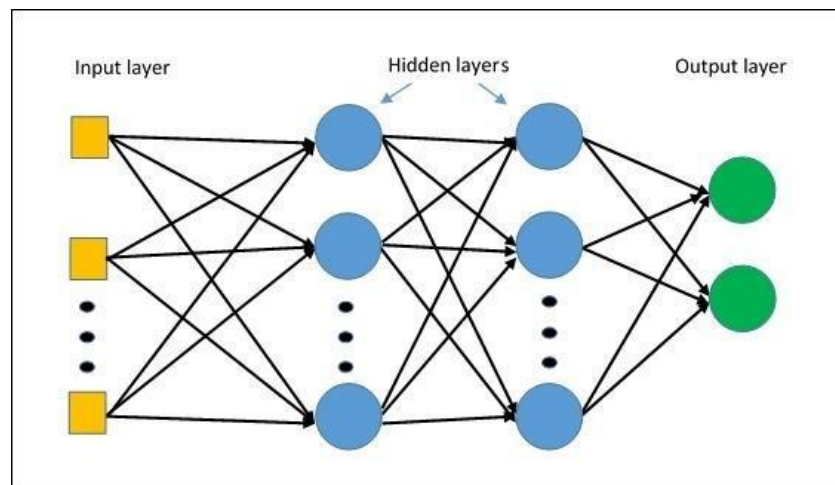
### Sigmoid

- The sigmoid function consists of 2 functions, logistic and tangential. The values of logistic function range from 0 and 1 and -1 to +1 for tangential function.

## Multilayer feed forward Network:

- A multilayer perceptron (MLP) is a class of feedforward artificial neural network. An MLP consists of at least three layers of nodes: an input layer, a hidden layer and an output layer.
- Except for the input nodes, each node is a neuron that uses a nonlinear activation function.
- MLP utilizes a supervised learning technique called backpropagation for training. Its multiple layers and non-linear activation distinguish MLP from a linear perceptron.

- It can



distinguish

data that is not linearly separable.

- Multilayer perceptrons are sometimes colloquially referred to as "vanilla" neural networks, especially when they have a single hidden layer.
- The term "multilayer perceptron" does not refer to a single perceptron that has multiple layers. Rather, it contains many perceptrons that are organized into layers. An alternative is "multilayer perceptron network".
- Moreover, MLP "perceptrons" are not perceptrons in the strictest possible sense. True perceptrons are formally a special case of artificial neurons that use a threshold activation function such as the Heaviside step function.

- MLP perceptrons can employ arbitrary activation functions. A true perceptron performs binary classification (either this or that), an MLP neuron is free to either perform classification or regression, depending upon its activation function.
- The term "multilayer perceptron" later was applied without respect to nature of the nodes/layers, which can be composed of arbitrarily defined artificial neurons, and not perceptrons specifically. This interpretation avoids the loosening of the definition of "perceptron" to mean an artificial neuron in general.

### Delta Rule:

The delta training rule is best understood by considering the task of training an un-thresholded perceptron; that is, a linear unit for which the output  $o$  is given by .

$$o(\vec{x}) = \vec{w} \cdot \vec{x}$$

Thus, a linear unit corresponds to the first stage of a perceptron, without the threshold.

Although there are many ways to define this error, one common measure that will turn out to be especially

convenient  $\frac{1}{2}$  —  
 $\frac{1}{2}$  is:

$$E[\vec{w}] \equiv \frac{1}{2} \sum_{d \in D} (t_d - o_d)^2$$

### Learning rate:

The size of gradient steps is called the *learning rate*. With a high learning rate we can cover more ground each step, but we risk overshooting the lowest point since the slope of the hill is constantly changing. With a very low learning rate, we can confidently move in the direction of the negative gradient since we are recalculating it so frequently. A low learning rate is more precise, but calculating the gradient is time-consuming, so it will take us a very long time to get to the bottom.

### Cost function

A [Loss Functions](#) tells us “how good” our model is at making predictions for a given set of parameters. The cost function has its own curve and its own gradients. The slope of this curve tells us how to update our parameters to make the model more accurate.

### Step-by-step

Now let's run gradient descent using our new cost function. There are two parameters in our cost function we can control:  $m$  (weight) and  $b$  (bias). Since we need to consider the impact each one has on the final prediction, we need to use partial derivatives. We calculate the partial derivatives of the cost function with respect to each parameter and store the results in a gradient. Math:

Given the cost function:

$$f(m, b) = \frac{1}{N} \sum_{i=1}^n (y_i - (mx_i + b))^2$$

The gradient can be calculated as:

$$f'(m, b) = \begin{bmatrix} \frac{df}{dm} \\ \frac{df}{db} \end{bmatrix} = \begin{bmatrix} \frac{1}{N} \sum -2x_i(y_i - (mx_i + b)) \\ \frac{1}{N} \sum -2(y_i - (mx_i + b)) \end{bmatrix}$$

To solve for the gradient, we iterate through our data points using our new  $m$  and  $b$  values and compute the partial derivatives. This new gradient tells us the slope of our cost function at our current position (current parameter values) and the direction we should move to update our parameters. The size of our update is controlled by the learning rate.

## Multilayer Artificial Neural Network:

A fully connected multi-layer neural network is called a Multilayer Perceptron (MLP).

Perceptron rule and Adaline rule were used to train a single-layer neural network. It has 3 layers including one hidden layer. If it has more than 1 hidden layer, it is called a deep ANN. An MLP is a typical example of a feedforward artificial neural network. In this figure, the  $i$ th activation unit in the  $l$ th layer is denoted as  $a_i(l)$ .

The number of layers and the number of neurons are referred to as hyperparameters of a neural network, and these need tuning. Cross-validation techniques must be used to find ideal values for these. The weight adjustment training is done via backpropagation. Deeper neural networks are better at processing data. However, deeper layers can lead to vanishing gradient problem. Special algorithms are required to solve this issue.

**Back-Propagation Algorithm working scenario:** This is a quite surprising fact, because it states that we know the form of the distribution that governs the sample mean ? even when we do not know the form of the underlying distribution that governs the individual  $Y_i$  that are being observed! Furthermore, the Central Limit Theorem describes how the mean and variance of  $Y$  can be used to determine the mean and variance of the individual  $Y_i$ . Back Propagation neural network

Multilayer neural networks use a most common technique from a variety of learning technique, called the back- propagation algorithm. In back propagation neural network, the output values are compared with the correct answer to compute the value of some predefined error function. By various techniques the error is then fed back through the network. Using this information, the algorithms adjust the weights of each connection in order to reduce the value of the error function by some small amount. After repeating this process for a sufficiently large number of training cycles the network will usually converge to some state where the error of the calculation is small.

The goal of back propagation, as with most training algorithms, is to iteratively adjust the weights in the network to produce the desired output by minimizing the output error. The algorithm's goal is to solve credit assignment problem. Back propagation is a gradient-descent approach in that it uses the minimization of first-order derivatives to find an optimal solution. The standard back propagation algorithm is given below.

**Step1:** Build a network with the chosen number of input, hidden and output units.

**Step2:** Initialize all the weights to low random values.

**Step3:** Randomly, choose a single training pair.

**Step4:** Copy the input pattern to the input layer.

**Step5:** Cycle the network so that the activation from the inputs generates the activations in the hidden and output layers.

**Step6:** Calculate the error derivative between the output activation and the final output.

**Step7:** Apply the method of back propagation to the summed products of the weights and errors in the output layer in order to calculate the error in the hidden units.

**Step8:** Update the weights attached the each unit according to the error in that unit, the output from the unit below it and the learning parameters, until the error is sufficiently low.

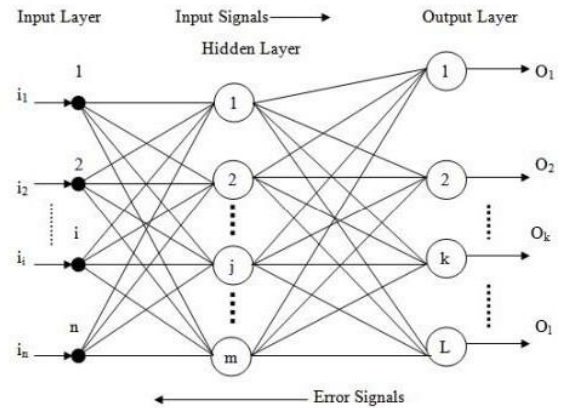


Figure Three layer back-propagation neural network

**Input:**

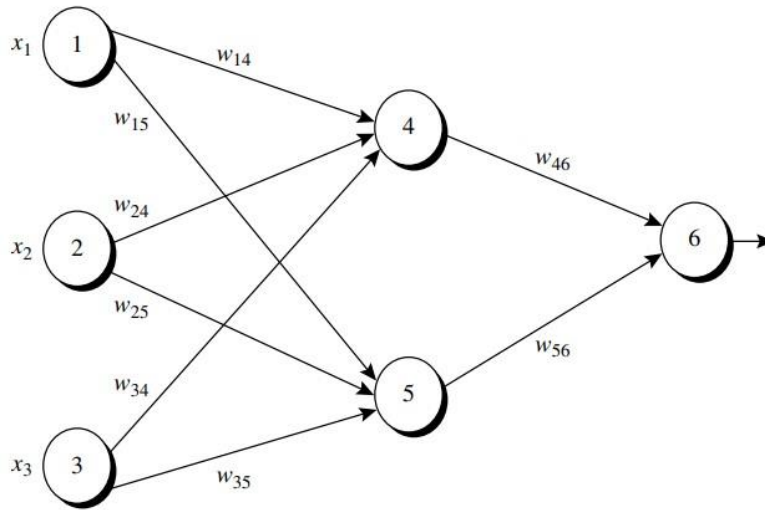
- $D$ , a data set consisting of the training tuples and their associated target values;
- $l$ , the learning rate;
- $network$ , a multilayer feed-forward network.

**Output:** A trained neural network.**Method:**

```

(1) Initialize all weights and biases in  $network$ ;
(2) while terminating condition is not satisfied {
(3)   for each training tuple  $X$  in  $D$  {
(4)     // Propagate the inputs forward:
(5)     for each input layer unit  $j$  {
(6)        $O_j = I_j$ ; // output of an input unit is its actual input value
(7)     for each hidden or output layer unit  $j$  {
(8)        $I_j = \sum_i w_{ij} O_i + \theta_j$ ; // compute the net input of unit  $j$  with respect to
        the previous layer,  $i$ 
(9)        $O_j = \frac{1}{1 + e^{-I_j}}$ ; } // compute the output of each unit  $j$ 
(10)    // Backpropagate the errors:
(11)    for each unit  $j$  in the output layer
(12)       $Err_j = O_j(1 - O_j)(T_j - O_j)$ ; // compute the error
(13)    for each unit  $j$  in the hidden layers, from the last to the first hidden layer
(14)       $Err_j = O_j(1 - O_j) \sum_k Err_k w_{jk}$ ; // compute the error with respect to
        the next higher layer,  $k$ 
(15)    for each weight  $w_{ij}$  in  $network$  {
(16)       $\Delta w_{ij} = (l) Err_j O_i$ ; // weight increment
(17)       $w_{ij} = w_{ij} + \Delta w_{ij}$ ; } // weight update
(18)    for each bias  $\theta_j$  in  $network$  {
(19)       $\Delta \theta_j = (l) Err_j$ ; // bias increment
(20)       $\theta_j = \theta_j + \Delta \theta_j$ ; } // bias update
(21)  } }
```

To derive the back propagation algorithm, let us consider the three layer network shown in figure.



**re 9.5** Example of a multilayer feed-forward neural network.

**le 9.1** Initial Input, Weight, and Bias Values

| $x_1$ | $x_2$ | $x_3$ | $w_{14}$ | $w_{15}$ | $w_{24}$ | $w_{25}$ | $w_{34}$ | $w_{35}$ | $w_{46}$ | $w_{56}$ | $\theta_4$ | $\theta_5$ | $\theta_6$ |
|-------|-------|-------|----------|----------|----------|----------|----------|----------|----------|----------|------------|------------|------------|
| 1     | 0     | 1     | 0.2      | -0.3     | 0.4      | 0.1      | -0.5     | 0.2      | -0.3     | -0.2     | -0.4       | 0.2        | 0.1        |



**Table 9.2** Net Input and Output Calculations

| <i>Unit, j</i> | <i>Net Input, I<sub>j</sub></i>               | <i>Output, O<sub>j</sub></i> |
|----------------|-----------------------------------------------|------------------------------|
| 4              | $0.2 + 0 - 0.5 - 0.4 = -0.7$                  | $1/(1 + e^{0.7}) = 0.332$    |
| 5              | $-0.3 + 0 + 0.2 + 0.2 = 0.1$                  | $1/(1 + e^{-0.1}) = 0.525$   |
| 6              | $(-0.3)(0.332) - (0.2)(0.525) + 0.1 = -0.105$ | $1/(1 + e^{0.105}) = 0.474$  |

**Table 9.3** Calculation of the Error at Each Node

| <i>Unit, j</i> | <i>Err<sub>j</sub></i>                       |
|----------------|----------------------------------------------|
| 6              | $(0.474)(1 - 0.474)(1 - 0.474) = 0.1311$     |
| 5              | $(0.525)(1 - 0.525)(0.1311)(-0.2) = -0.0065$ |
| 4              | $(0.332)(1 - 0.332)(0.1311)(-0.3) = -0.0087$ |

**Table 9.4** Calculations for Weight and Bias Updating

| <i>Weight<br/>or Bias</i> | <i>New Value</i>                       |
|---------------------------|----------------------------------------|
| $w_{46}$                  | $-0.3 + (0.9)(0.1311)(0.332) = -0.261$ |
| $w_{56}$                  | $-0.2 + (0.9)(0.1311)(0.525) = -0.138$ |
| $w_{14}$                  | $0.2 + (0.9)(-0.0087)(1) = 0.192$      |
| $w_{15}$                  | $-0.3 + (0.9)(-0.0065)(1) = -0.306$    |
| $w_{24}$                  | $0.4 + (0.9)(-0.0087)(0) = 0.4$        |
| $w_{25}$                  | $0.1 + (0.9)(-0.0065)(0) = 0.1$        |
| $w_{34}$                  | $-0.5 + (0.9)(-0.0087)(1) = -0.508$    |
| $w_{35}$                  | $0.2 + (0.9)(-0.0065)(1) = 0.194$      |
| $\theta_6$                | $0.1 + (0.9)(0.1311) = 0.218$          |
| $\theta_5$                | $0.2 + (0.9)(-0.0065) = 0.194$         |
| $\theta_4$                | $-0.4 + (0.9)(-0.0087) = -0.408$       |